



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS FLORIANÓPOLIS
CURSO DE GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Lucas Bueno Cesario

**POLYGLOT IMPORT CSV: UMA SOLUÇÃO PARA IMPORTAÇÃO DE
DOCUMENTOS CSV PARA MÚLTIPLOS BANCOS DE DADOS**

Florianópolis
2026

Lucas Bueno Cesario

**POLYGLOT IMPORT CSV: UMA SOLUÇÃO PARA IMPORTAÇÃO DE
DOCUMENTOS CSV PARA MÚLTIPLOS BANCOS DE DADOS**

Trabalho de Conclusão de Curso submetido ao Curso de Graduação em Ciência da Computação do Centro Tecnológico da Universidade Federal de Santa Catarina para obtenção do título de Bacharel em Ciência da Computação.

Orientador: Prof. Ronaldo dos Santos Mello, Dr.

Florianópolis
2026

Ficha catalográfica para trabalhos acadêmicos.

Cesario, Lucas Bueno

Polyglot Import CSV : Uma solução para importação de documentos CSV para múltiplos bancos de dados / Lucas Bueno Cesario ; orientador, Ronaldo dos Santos Mello, 2026.

49 p.

Trabalho de Conclusão de Curso (graduação) - Universidade Federal de Santa Catarina, Centro Tecnológico, Graduação em Ciências da Computação, Florianópolis, 2026.

Inclui referências.

1. Ciências da Computação. 2. Persistência poliglota. 3. Importação de dados CSV. 4. NoSQL. 5. Software Livre. I. Mello, Ronaldo dos Santos. II. Universidade Federal de Santa Catarina. Graduação em Ciências da Computação. III. Título.

Lucas Bueno Cesario

**POLYGLOT IMPORT CSV: UMA SOLUÇÃO PARA IMPORTAÇÃO DE
DOCUMENTOS CSV PARA MÚLTIPLOS BANCOS DE DADOS**

Este Trabalho de Conclusão de Curso foi julgado adequado para obtenção do Título de “Bacharel em Ciência da Computação” e aprovado em sua forma final pelo Curso de Graduação em Ciências da Computação.

Florianópolis, _____ de _____ de 2026.

Profa. Lúcia Helena Martins Pacheco, Dra.
Coordenadora do Curso

Banca Examinadora:

Prof. Ronaldo dos Santos Mello, Dr.
Orientador
Universidade Federal de Santa Catarina

Profa. Carina Friedrich Dorneles, Dra.
Avaliadora
Universidade Federal de Santa Catarina

Prof. Jônata Tyska Carvalho, Dr.
Avaliador
Universidade Federal de Santa Catarina

RESUMO

A persistência poliglota é essencial na arquitetura contemporânea de bancos de dados, integrando diversas tecnologias de armazenamento em uma mesma aplicação. A ferramenta *Polyglot Import CSV* destaca-se como solução para otimizar a importação de dados CSV em diversos bancos de dados simultaneamente, superando desafios da persistência poliglota. Optou-se pelo formato CSV por ser amplamente utilizado na importação de dados pelos diversos SGBDs e por ser um formato simples e familiar tanto a humanos quanto a sistemas. Por meio de configurações declarativas em arquivos JSON, a ferramenta oferece suporte ao principal representante de código aberto dos bancos de dados relacionais — o PostgreSQL — e aos principais representantes dos bancos de dados NoSQL: Redis (chave-valor), MongoDB (documento), Apache Cassandra (colunar) e Neo4j (grafo). Não foram identificadas ferramentas equivalentes que realizem, a partir de um único arquivo CSV e de regras declarativas, a importação simultânea para múltiplos paradigmas de dados, o que evidencia a originalidade da proposta. A publicação do código aberto da ferramenta impulsiona avanços colaborativos na dinâmica arquitetura moderna de bancos de dados.

Palavras-chave: Persistência poliglota. Ferramenta *Polyglot Import CSV*. Importação de dados CSV. JSON. SQL. NoSQL. Software Livre.

ABSTRACT

Polyglot persistence is essential in contemporary database architecture, integrating various storage technologies within the same application. The Polyglot Import CSV tool stands out as a solution for optimizing CSV data import into multiple databases simultaneously, overcoming polyglot persistence challenges. The CSV format was chosen because it is widely used for data import by various DBMSs and because it is a simple and familiar format for both humans and systems. Through declarative configurations in JSON files, the tool supports the leading open-source representative of relational databases — PostgreSQL — and the main representatives of NoSQL databases: Redis (key-value), MongoDB (document), Apache Cassandra (columnar), and Neo4j (graph). No equivalent tools were identified that perform, from a single CSV file and declarative rules, the simultaneous import to multiple data paradigms, which highlights the originality of the proposal. The open-source publication of the tool drives collaborative advances in the dynamic modern database architecture.

Keywords: *Polyglot persistence. Polyglot Import CSV tool. CSV data import. JSON. SQL. NoSQL. Free Software.*

LISTA DE ILUSTRAÇÕES

Figura 1 – Exemplo de aplicação da Persistência Poliglota em um e-commerce.	15
Figura 2 – Diferentes tipos de modelos de dados NoSQL.	16
Figura 3 – Modelo chave-valor: chaves apontando para valores arbitrários (Redis).	17
Figura 4 – Modelo colunar: chaves de linha e famílias de colunas (Cassandra).	18
Figura 5 – Modelo de documento: documentos autocontidos com campos variáveis (MongoDB).	19
Figura 6 – Modelo de grafo: nós e arestas tipadas com propriedades (Neo4j). .	20
Figura 7 – Visão geral da ferramenta: entradas, processamento e saídas. . . .	28
Figura 8 – Estrutura do JSON Schema de importação (import_config).	30
Figura 9 – Estrutura do JSON Schema de conexão (sgbd_config).	30
Figura 10 – Fluxo de carregamento e validação dos dois arquivos de configuração.	39
Figura 11 – Algoritmo de execução da importação Polyglot Import CSV.	40

LISTA DE TABELAS

Tabela 1 – Campos de mapeamento de coluna (columnSpec).	32
Tabela 2 – Linhas action == stock do ecommerce_join.csv (colunas de identificação resumidas).	43
Tabela 3 – Relação inventory materializada no PostgreSQL a partir das linhas de estoque.	44
Tabela 4 – Partição do usuário user1 na tabela user_activity_log (Cassandra).	45
Tabela 5 – Materialização de uma linha action == stock nos diferentes SGBDs.	46
Tabela 6 – Contagens reportadas pelo modo --dry-run no cenário e-commerce.	47
Tabela 7 – Grupos de colunas do ecommerce_join.csv preenchidos por ação.	60
Tabela 8 – Usuários do ecommerce_join.csv e seus dados de identificação e endereço.	61
Tabela 9 – Linhas action == stock do ecommerce_join.csv.	62
Tabela 10 – Linhas action == purchase do ecommerce_join.csv.	63
Tabela 11 – Linhas action == select_product do ecommerce_join.csv.	63
Tabela 12 – Linhas action == add_to_cart do ecommerce_join.csv.	64

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
API	<i>Application Programming Interface</i> (Interface de Programação de Aplicações)
BSON	<i>Binary JSON</i>
CLI	<i>Command Line Interface</i> (Interface de Linha de Comando)
CSV	<i>Comma-Separated Values</i> (Valores Separados por Vírgula)
DDL	<i>Data Definition Language</i> (Linguagem de Definição de Dados)
JSON	<i>JavaScript Object Notation</i>
NoSQL	<i>Not Only SQL</i>
SGBD	Sistema de Gerenciamento de Banco de Dados
SGPD	Sistema de Gerenciamento Poliglota de Dados
SQL	<i>Structured Query Language</i> (Linguagem de Consulta Estruturada)
TCC	Trabalho de Conclusão de Curso
UFSC	Universidade Federal de Santa Catarina
XML	<i>Extensible Markup Language</i>

SUMÁRIO

1	INTRODUÇÃO	11
1.1	Visão Geral	11
1.2	Objetivos	12
1.2.1	Objetivo Geral	12
1.2.2	Objetivos Específicos	12
1.3	Metodologia	12
1.4	Estrutura do Trabalho	13
2	FUNDAMENTAÇÃO TEÓRICA	14
2.1	Persistência Poliglota	14
2.2	Bancos de Dados NoSQL	16
2.2.1	Chave-valor	16
2.2.2	Colunar	17
2.2.3	Documento	18
2.2.4	Grafo	19
2.3	Bancos de Dados Multimodelo e Arquiteturas de Persistência Poliglota	20
3	TRABALHOS RELACIONADOS	22
3.1	Importação de Arquivo CSV para um Único SGBD	22
3.1.1	PostgreSQL	22
3.1.2	Redis	23
3.1.3	MongoDB	24
3.1.4	Cassandra	24
3.1.5	Neo4j	25
3.2	Importação de Arquivo CSV para SGBD Multimodelo	25
3.3	Considerações sobre os Trabalhos Relacionados	26
4	A PROPOSTA DA FERRAMENTA POLYGLOTIMPORTCSV	28
4.1	Visão Geral	28
4.2	Formato de Configuração (JSON Schema)	29
4.2.1	Visão Geral da Estrutura dos Arquivos de Configuração	29
4.2.2	Raiz e Elementos Comuns	31
4.2.3	PostgreSQL	32
4.2.4	MongoDB	34
4.2.5	Apache Cassandra	35

4.2.6	Redis	36
4.2.7	Neo4j	37
4.2.8	Validação, Consistência entre Arquivos e Fluxo de Carregamento . .	38
4.3	Algoritmo de Execução da Importação	39
4.3.1	O Cenário de Exemplo e o Arquivo de Dados	42
4.3.2	A Importação em Cada SGBD	43
4.3.3	Exemplo de Uso e Evidência de Execução (Cenário E-commerce) . .	47
5	ATIVIDADES FUTURAS	49
	REFERÊNCIAS	50
	APÊNDICES	52
	APÊNDICE A – JSON SCHEMAS DE CONFIGURAÇÃO NA FORMA SERIALIZADA	53
A.1	Schema de Importação (import_config.schema.json)	53
A.2	Schema de Conexão (sgbd_config.schema.json)	56
	APÊNDICE B – O ARQUIVO DE DADOS ECOMMERCE_JOIN.CSV . . .	59
B.1	Linhas com action == stock	61
B.2	Linhas com action == purchase	62
B.3	Linhas com action == select_product	63
B.4	Linhas com action == add_to_cart	64
	APÊNDICE C – ARQUIVOS DE CONFIGURAÇÃO DO CENÁRIO E-COMMERCE	65
C.1	Arquivo de Importação (import_config.json)	65
C.2	Arquivo de Conexão (sgbd_config.json)	68

1 INTRODUÇÃO

1.1 VISÃO GERAL

Nas esferas industrial e acadêmica de bancos de dados, a concepção de que um único modelo de dados não atende a todas as necessidades tem conquistado aceitação, apontando para a perspectiva de persistência poliglota no futuro. Embora os sistemas de banco de dados relacionais devam manter sua predominância, espera-se uma convivência com diversas formas de sistemas, como os NoSQL ([SADALAGE; FOWLER, 2013](#)).

Algumas pesquisas têm sido feitas na área de persistência poliglota, como sugestões de modelos de dados unificados ([CHILLÓN et al., 2023](#)), comparação de desempenho de bancos de dados multi-modelos contra abordagens de persistência poliglota ([YE et al., 2023](#)), revisões sistemáticas sobre modelagem poliglota de dados ([SILVA; BORNIA; MELLO, 2024](#)), tutoriais sobre o estado da arte e desafios abertos ([KIEHN et al., 2022](#)), e a adoção da persistência poliglota em Big Data ([KHINE; WANG, 2019](#)).

Dada essa necessidade crescente da adoção de persistência poliglota pelas aplicações, percebe-se a falta de uma ferramenta única de importação simultânea de dados para diversos SGBDs que tratem modelos de dados diferentes. Até onde se investigou (Capítulo 3), não foram identificadas ferramentas equivalentes: os importadores nativos atendem a um único SGBD por vez, e as soluções de migração ou de modelagem poliglota não realizam a materialização simultânea de um mesmo CSV em múltiplos paradigmas de dados — o que ressalta a originalidade desta proposta.

Assim sendo, a intenção deste trabalho é desenvolver a ferramenta *Polyglot Import CSV*, que importa dados armazenados em CSV para SGBDs relacionais e NoSQL. A configuração da ferramenta será baseada em dois arquivos JSON: um de importação, mapeando entidades e relacionamentos em cada SGBD, considerado seu modelo de dados; outro de configuração de conexão dos SGBDs envolvidos. A escolha pelo formato JSON deve-se à sua simplicidade e ampla adoção; sua finalidade é detalhar, de forma declarativa, o mapeamento das colunas do CSV para os modelos de dados de cada SGBD, permitindo a validação prévia da importação.

CSV (*Comma-Separated Values*) é um arquivo de valores separados por vírgulas, frequentemente visualizado em Excel e outras planilhas eletrônicas. Pode haver outros tipos de valores como delimitadores, mas o mais comum é a vírgula. Muitos sistemas e processos hoje convertem seus dados para o formato CSV para saídas de

arquivo para outros sistemas, relatórios amigáveis para humanos, disponibilização de dados públicos, e outras necessidades. É um formato de arquivo padrão com o qual humanos e sistemas já estão familiarizados ao usar e manipular.

1.2 OBJETIVOS

1.2.1 Objetivo Geral

Desenvolver a ferramenta *Polyglot Import CSV* que possibilitará a importação de dados em arquivos CSV para múltiplos SGBDs: PostgreSQL, Redis, MongoDB, Cassandra ou Neo4j.

1.2.2 Objetivos Específicos

- Projetar os arquivos de configuração JSON para que eles sigam sintaxes específicas, e flexíveis para lidar com vários modelos de dados, além de adicionar opções como filtros, escolha de chave primária e escolha do nome da coluna no banco de dados;
- Não permitir configurações incorretas, ou seja, se houver erro em algum lugar dos arquivos de configuração, a importação não deve começar até que sejam corrigidos (validação prévia da configuração);
- Avaliar o desempenho da ferramenta para diferentes volumes e modelos de dados destino;
- Disponibilizar a ferramenta de forma acessível ao usuário: inicialmente por meio de uma interface de linha de comando (CLI), com possibilidade de evolução para uma interface gráfica em trabalhos futuros.

1.3 METODOLOGIA

O desenvolvimento deste trabalho segue uma metodologia de pesquisa aplicada, adotando as seguintes etapas:

1. Estudo sobre persistência poliglota e seu estado da arte;
2. Estudo sobre os paradigmas não-relacionais que serão abrangidos pela solução: chave-valor, documento, grafo e colunar;
3. Estudo de decisão de linguagem de implementação que demonstra melhor produtividade para codificar a solução;
4. Projeto dos arquivos de configuração JSON;

5. Projeto da instrução principal de importação que utiliza os arquivos de configuração;
6. Implementação da solução “Polyglot Import CSV”;
7. Testes da solução “Polyglot Import CSV” implementada.

Na primeira etapa, foi estudado o que é a persistência poliglota, e a sua relevância no cenário atual.

Na segunda etapa, foram estudados os diversos modelos de dados não-relacionais: como cada modelo representa suas entidades; se há relacionamentos entre entidades no modelo; se é possível a existência de entidades aninhadas e multivaloradas; as limitações de cada modelo.

Na terceira etapa, foram estudadas as formas de implementar a solução usando as linguagens Java e Python e, pela simplicidade e produtividade, a linguagem Python foi escolhida.

Na quarta e quinta etapas, foram projetados os dois arquivos de configuração JSON — um para o mapeamento da importação (entidades, relacionamentos e colunas) e outro para a conexão dos SGBDs — bem como a instrução principal de importação que os utiliza.

As últimas etapas, que estão em andamento, tratam da implementação da solução proposta, “Polyglot Import CSV”, e dos testes da mesma com diferentes volumes de dados CSV e importações simples e complexas para diferentes modelos de dados destino.

1.4 ESTRUTURA DO TRABALHO

Este trabalho está estruturado em cinco capítulos principais. O primeiro aborda os objetivos, a metodologia e o contexto que levaram à elaboração deste TCC. O capítulo 2 revisa fundamentos teóricos. O capítulo 3 apresenta trabalhos relacionados. O capítulo 4 descreve a proposta da ferramenta *Polyglot Import CSV* e o capítulo 5 as atividades futuras (TCC II).

2 FUNDAMENTAÇÃO TEÓRICA

2.1 PERSISTÊNCIA POLIGLOTA

Ford (2008) introduziu o conceito de “programação poliglota” para expressar a ideia de que aplicativos devem ser desenvolvidos utilizando uma combinação de linguagens, aproveitando o fato de que diferentes linguagens são mais adequadas para lidar com diferentes problemas. Uma das consequências notáveis da programação poliglota é a transição para a “persistência poliglota”, usando a mesma analogia: pode-se dizer que múltiplas tecnologias de bancos de dados podem ser utilizadas como armazenamento persistente para diferentes tipos de casos de uso. Embora ainda exista uma quantidade considerável de dados gerenciados por sistemas relacionais, a abordagem predominante passa a ser a reflexão sobre como se deseja manipular os dados antes de determinar qual tecnologia é a mais adequada para essa manipulação (FOWLER, 2011).

A persistência poliglota abrange uma variedade de formatos de dados, incluindo estruturados, semi-estruturados e não estruturados. Dados estruturados geralmente englobam formatos como relacional, objeto-relacional, e grafos de propriedades. Dados semi-estruturados incluem principalmente documentos JSON e XML. Por fim, dados não estruturados são representados por arquivos de texto, imagens e áudios, dentre outros (YE et al., 2023).

Um domínio em que a persistência poliglota pode ser aplicada é o de um e-commerce. Nesse cenário, diferentes categorias de dados possuem características e padrões de acesso distintos — volume de escrita, frequência de leitura, necessidade de consistência, estrutura dos registros — o que torna mais adequado o uso de um SGBD específico para cada uma delas. A Figura 1 ilustra esse cenário, mapeando cada categoria de dado do e-commerce ao SGBD mais adequado, conforme sugerido a seguir:

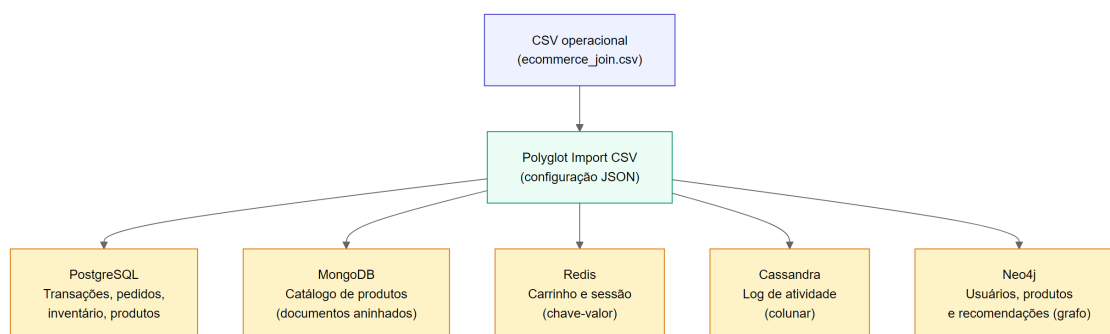


Figura 1 – Exemplo de aplicação da Persistência Poliglota em um e-commerce.

Fonte: Elaborado pelo autor (2026).

- **Dados Financeiros/Transacionais de Pagamento e Inventário de Produtos:** Pedidos, pagamentos e estoques exigem garantias transacionais fortes (ACID) para evitar inconsistências — por exemplo, debitar um pagamento sem registrar o pedido correspondente. Além disso, consultas analíticas sobre esses dados frequentemente combinam várias entidades por meio de junções. Bancos de dados relacionais como o PostgreSQL são a escolha natural para esse perfil, pois oferecem suporte nativo a transações com consistência forte, integridade referencial e uma linguagem de consulta expressiva (SQL).
- **Dados do Catálogo de Produtos:** O catálogo de um e-commerce tende a ter esquema variável: diferentes produtos possuem atributos distintos (um livro físico com ISBN e editora; um eletrodoméstico com voltagem e garantia). Além disso, o catálogo é lido com muito maior frequência do que escrito, e cada consulta normalmente recupera o produto completo de uma vez. Bancos de dados orientados a documentos como o MongoDB são otimizados exatamente para esse perfil: armazenam cada produto como um documento autocontido (favorecendo leituras de agregados complexos), suportam esquemas flexíveis e oferecem índices secundários e *pipelines* de agregação para fins de consulta.
- **Dados da Sessão e do Carrinho de Compras do Usuário:** Dados de sessão e carrinho são acessados a cada interação do usuário durante a navegação, exigindo latência mínima, e são também inerentemente temporários — um carrinho abandonado não precisa persistir indefinidamente. Bancos de dados chave-valor em memória como o Redis oferecem latência de microssegundos, expiração automática de chaves e são horizontalmente escaláveis para absorver picos de tráfego, tornando-se a escolha ideal para dados de natureza efêmera e de acesso intenso.
- **Dados de Atividade do Usuário:** Eventos de navegação (*clickstream*), visualizações de produto e histórico de ações são gerados em alto volume e alta

velocidade. Consultas típicas percorrem o histórico de um usuário em um intervalo de tempo, ou agregam eventos por tipo de ação. Bancos de dados colunares como o Apache Cassandra são projetados para ingestão massiva com particionamento por chave primária composta (por exemplo, `(user_id, timestamp)`), garantindo escritas rápidas e leituras eficientes por usuário ou período — padrão ideal para séries temporais de atividade.

- **Dados de Recomendação:** Sistemas de recomendação dependem de relacionamentos entre entidades: usuários que compraram os mesmos produtos, produtos frequentemente adquiridos em conjunto ou usuários com perfis semelhantes. Esses padrões são naturalmente modelados como grafos, onde nós representam usuários e produtos, e arestas representam interações (comprou, avaliou, visualizou, etc.). Bancos de dados orientados a grafos como o Neo4j permitem percorrer esses relacionamentos com eficiência por meio de consultas Cypher, tarefa que em um banco de dados relacional exigiria múltiplas junções e difícil otimização.

2.2 BANCOS DE DADOS NOSQL

Os bancos de dados NoSQL podem ser divididos em subcategorias com base em seus modelos de dados. Este trabalho utiliza a classificação de [Hecht e Jablonski \(2011\)](#), que categoriza os bancos de dados NoSQL em quatro tipos: chave-valor, colunares, documentos e grafos.

A Figura 2 ilustra esses modelos de dados, que são detalhados a seguir.

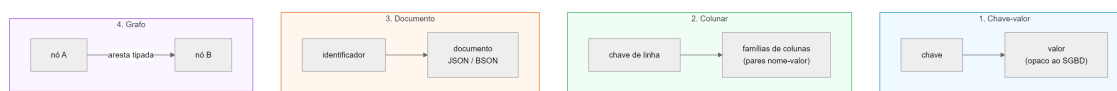


Figura 2 – Diferentes tipos de modelos de dados NoSQL.

Fonte: Elaborado pelo autor (2026).

2.2.1 Chave-valor

Os bancos de dados de chave-valor têm um modelo de dados simples baseado justamente em pares chave-valor, semelhante a um mapa associativo ou dicionário. A chave identifica exclusivamente o valor e é usada para armazenar e recuperar o valor no banco de dados, funcionando como uma chave primária. O valor pode ser usado para armazenar qualquer dado arbitrário, incluindo um número inteiro, uma *string*, um *array* ou um objeto, proporcionando uma representação de dados sem esquema conhecido.

Além disso, os bancos de dados de chave-valor são muito eficientes no armazenamento de dados distribuídos, mas não são adequados para cenários que requerem a definição de relacionamentos entre dados ou esquemas fortemente estruturados.

Qualquer funcionalidade que necessite desses requisitos deve ser implementada na aplicação cliente que interage com o banco de dados chave-valor.

Como os valores são opacos para os bancos de dados, esses não podem lidar com consultas e indexações a nível de dados, podendo realizar consultas apenas por meio de chaves. Um exemplo de SGBD chave-valor é o Redis, que mantém os dados tanto na memória como no disco. A Figura 3 ilustra o modelo com um exemplo de carrinho de compras e sessão de usuário, em que cada chave aponta para um valor opaco ao SGBD.

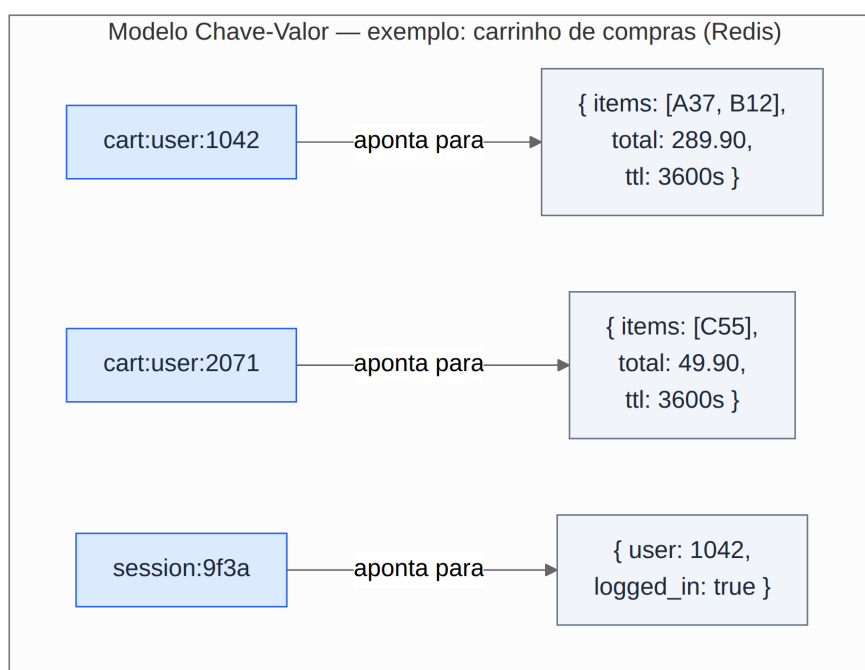


Figura 3 – Modelo chave-valor: chaves apontando para valores arbitrários (Redis).

Fonte: Elaborado pelo autor (2026).

2.2.2 Colunar

Em bancos de dados colunares, o conjunto de dados consiste em várias linhas, cada uma identificada por uma chave de linha única, semelhante a uma chave primária. Cada linha é composta por uma família (ou conjunto) de colunas, e diferentes linhas podem ter diferentes famílias de colunas. Semelhante aos bancos de dados de chave-valor, a chave de linha funciona como a chave, e o conjunto de famílias de colunas atua como o valor representado pela chave de linha. No entanto, colunas em uma família de colunas consistem em pares atributo-valor que podem ser indexadas. O SGBD Cassandra oferece a funcionalidade adicional de supercolunas, que são criadas agrupando várias colunas.

Normalmente, os dados pertencentes a uma linha são armazenados juntos no mesmo nó do servidor. No entanto, o Cassandra pode distribuir uma única linha

por vários nós de servidor usando chaves de partição compostas. Nos bancos de dados colunares, a configuração das famílias de colunas é tipicamente feita durante a inicialização. No entanto, a definição prévia de colunas não é necessária, oferecendo grande flexibilidade no armazenamento de qualquer tipo de dado.

De forma semelhante aos bancos de dados de chave-valor, qualquer lógica que exija relações deve ser implementada na aplicação cliente, ou seja, essa categoria de banco de dados também não têm ciência de relacionamentos entre dados e controle de integridade referencial. A Figura 4 apresenta um exemplo de *log* de atividade, em que cada chave de linha (*user_id*) agrupa uma família de colunas com eventos indexados por *timestamp*.

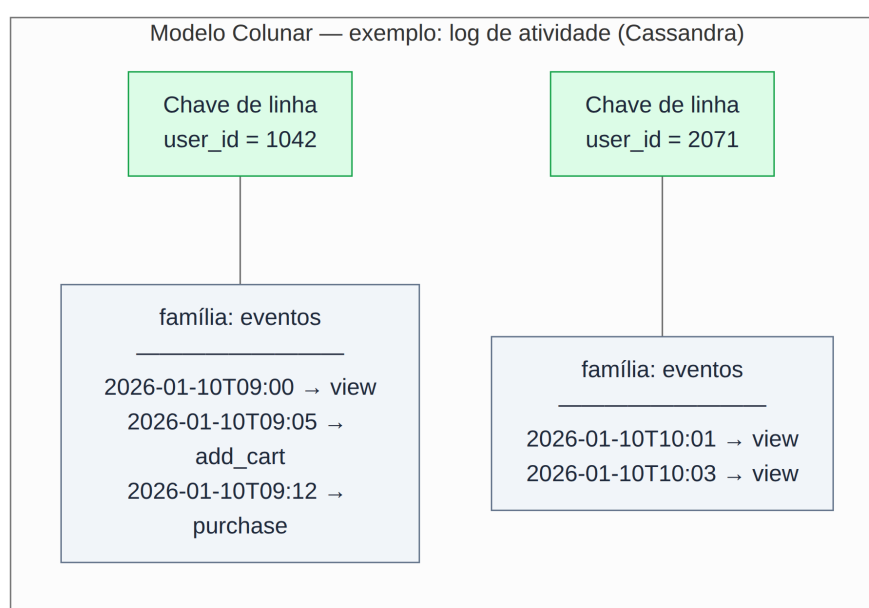


Figura 4 – Modelo colunar: chaves de linha e famílias de colunas (Cassandra).

Fonte: Elaborado pelo autor (2026).

2.2.3 Documento

Bancos de dados orientados a documento armazenam dados como documentos autocontidos, em geral em formato JSON ou BSON. Cada documento possui um identificador e um conjunto de campos que podem variar entre documentos de uma mesma coleção, favorecendo esquemas flexíveis e agregados de leitura (*read-optimized*). O SGBD MongoDB é um exemplo popular nesta categoria. Ele define coleções de documentos, índices secundários e *pipelines* de agregação, permitindo consultas ricas sem exigir junções (*joins*) no servidor. A Figura 5 ilustra uma coleção de catálogo de produtos, em que cada documento é autocontido e pode conter subdocumentos aninhados (como *category* e *stock*) e campos distintos entre si.

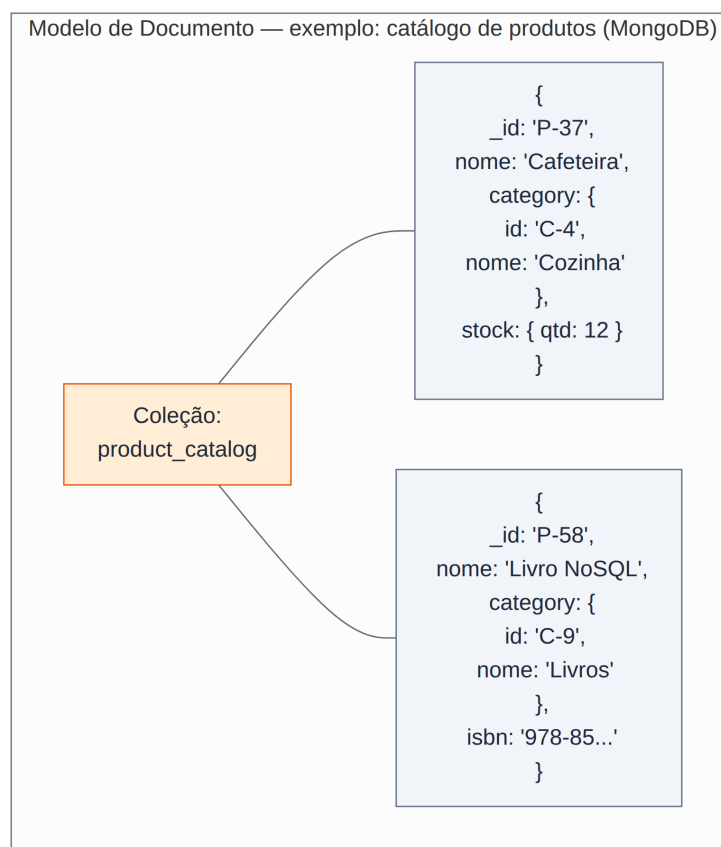


Figura 5 – Modelo de documento: documentos autocontidos com campos variáveis (MongoDB).

Fonte: Elaborado pelo autor (2026).

2.2.4 Grafo

Bancos de dados em grafo representam dados como **nós** (entidades) e **arestas** (relacionamentos tipados), com propriedades em ambos. Consultas exploram caminhos e vizinhanças de forma natural, o que é útil para recomendação, detecção de fraude e redes sociais. O Neo4j utiliza a linguagem Cypher para MERGE/CREATE de nós e relacionamentos.

Na ferramenta proposta, o destino grafo exige que o usuário declare explicitamente quais colunas do CSV identificam cada rótulo de nó e quais colunas alimentam propriedades do relacionamento (por exemplo, comprador → vendedor com atributos do pedido). A Figura 6 exemplifica um cenário de recomendação, em que nós User e Product são conectados por arestas tipadas (PURCHASED, VIEWED, SIMILAR_TO), algumas portando propriedades próprias.

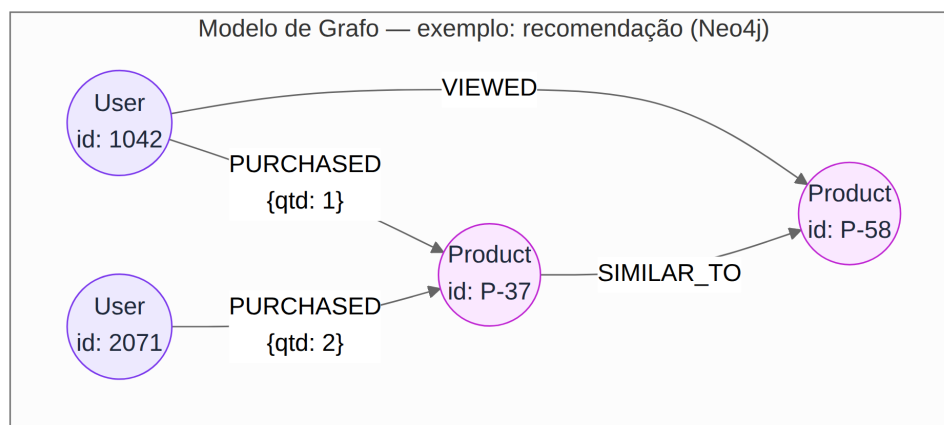


Figura 6 – Modelo de grafo: nós e arestas tipadas com propriedades (Neo4j).

Fonte: Elaborado pelo autor (2026).

2.3 BANCOS DE DADOS MULTIMODELO E ARQUITETURAS DE PERSISTÊNCIA POLIGLOTA

SGBDs multimodelo integram mais de um paradigma de dados no mesmo motor de gerência de dados (por exemplo, documento + chave-valor + grafo), oferecendo em geral uma interface de acesso unificada — como a linguagem AQL do SGBD ArangoDB, que combina filtros sobre estruturas de documento similares ao JSON e travessias em grafos em uma única abstração de modelo de dados que combina ambos os modelos NoSQL.

A persistência poliglota, por outro lado, combina vários SGBDs heterogêneos, cada um escolhido pela adequação a um caso de uso da aplicação (SADALAGE; FOWLER, 2013). Na literatura recente, essa ideia materializa-se como um SGPD (Sistema de Gerenciamento Poliglota de Dados): coordenação transparente entre tecnologias distintas, com requisitos de modelagem e expressividade de acesso (diferentes padrões de consulta) e capacidade de adaptação quando o *workload* evolui (KIEHN et al., 2022; GLAKE et al., 2022).

Tan et al. (2017) apresentam uma taxonomia útil para distinguir arquiteturas relacionadas: BD federado (fontes homogêneas, interface única), BD multistore (fontes heterogêneas, modelo canônico de consulta), BD *polystore* (fontes heterogêneas, múltiplas interfaces de acesso — exemplificadas por sistemas como o BigDAWG) e abordagens multimodelo (vários modelos no mesmo motor).

A escolha entre essas arquiteturas não é trivial. Roy-Hubara, Shoval e Sturm (2022) propõem critérios para selecionar quais modelos de dados usar em aplicações de persistência poliglota, destacando o *trade-off* entre especialização por SGBD e simplicidade operacional de um motor multimodelo.

O *Polyglot Import CSV* adota explicitamente a linha da persistência poliglota ao invés de multimodelo, conforme detalhado no capítulo 4. O usuário direciona entidades e filtros para PostgreSQL, Redis, MongoDB, Cassandra e Neo4j conforme o padrão de representação e acesso desejados.

3 TRABALHOS RELACIONADOS

Este capítulo apresenta e discute ferramentas oferecidas na indústria por SGBDs e também propostas na literatura acadêmica relacionadas com a importação de dados por SGBDs heterogêneos. A preferência está em dados no formato CSV que devem ser importados por alguma tecnologia de banco de dados, que é o foco deste trabalho.

3.1 IMPORTAÇÃO DE ARQUIVO CSV PARA UM ÚNICO SGBD

Esta seção investiga mecanismos de importação de dados presentes em SGBDs populares na indústria que são considerados neste trabalho.

3.1.1 PostgreSQL

No SGBD PostgreSQL você utiliza o comando `COPY` para realizar importações de dados CSV ([The PostgreSQL Global Development Group, 2026](#)). É necessário especificar a tabela com os nomes das colunas após a cláusula `COPY`. A ordem das colunas deve ser a mesma que aquelas no arquivo CSV, conforme ilustra o exemplo a seguir:

```
COPY nome_da_tabela(col1, col2, col3)
FROM 'C:/caminho_do_arquivo/dados_de_amostra.csv'
DELIMITER ','
CSV HEADER;
```

Caso o arquivo CSV contenha todas as colunas da tabela, você não precisa especificá-las explicitamente:

```
COPY nome_da_tabela
FROM 'C:/caminho_do_arquivo/dados_de_amostra.csv'
DELIMITER ','
CSV HEADER;
```

Você também deve inserir o caminho do arquivo CSV após a palavra-chave `FROM`. Como o formato do arquivo é CSV, você precisa especificar `DELIMITER`, bem como a cláusula `CSV`. A cláusula `HEADER` deve ser especificada para indicar que o arquivo CSV contém um cabeçalho. Quando o comando `COPY` importa dados, ele ignora o cabeçalho do arquivo.

O arquivo CSV deve ser lido diretamente pelo servidor PostgreSQL, não pela aplicação cliente. Portanto, ele deve ser acessível pela máquina do servidor PostgreSQL. Além disso, é necessário ter permissão de superusuário para executar com sucesso o comando `COPY`.

3.1.2 Redis

A maneira preferida de importar uma massa de dados para o Redis é gerar um arquivo de texto contendo o protocolo Redis, em formato bruto, para chamar os comandos necessários para inserir os dados requeridos ([Redis Ltd., 2026](#)).

Por exemplo, se é necessário gerar um grande conjunto de dados com bilhões de chaves no formato “chaveN → ValorN”, deve ser criado um arquivo contendo os seguintes comandos:

```
SET Chave0 Valor0
SET Chave1 Valor1
SET ChaveN ValorN
```

Uma vez criado esse arquivo, a ação restante é alimentá-lo no Redis. No passado, a maneira de fazer isso era usar o netcat com o seguinte comando:

```
(cat dados.txt; sleep 10) | nc localhost 6379
```

Nas versões 2.6 ou posteriores do Redis, o utilitário `redis-cli` suporta um novo modo chamado modo *pipe* que foi projetado para realizar o carregamento em massa. O comando a ser executado neste caso é o seguinte:

```
cat dados.txt | redis-cli --pipe
```

O utilitário `redis-cli` imprime um pequeno resumo. A saída típica é semelhante a esta:

```
All data transferred. Waiting for the last reply...
Last reply received from server.
errors: 0, replies: 1000000
```

A linha de contagem ao final é a informação mais importante do resumo: `errors` indica quantos comandos foram rejeitados pelo servidor, e `replies` indica o total de respostas recebidas. Se `errors: 0`, todos os comandos foram processados com sucesso. Um valor maior que zero em `errors` significa que ao menos um comando falhou — seja por sintaxe inválida no arquivo, por tipo de dado incompatível com a chave existente, ou por qualquer outra razão que o Redis tenha rejeitado.

Vale notar que o `--pipe` não exibe cada resposta individualmente (como um `+OK` por `SET`), o que é intencional: imprimir bilhões de respostas destruiria o ganho de *performance* do modo *pipe* e inundaria o terminal. Em vez disso, o `redis-cli` lê e contabiliza todas as respostas silenciosamente conforme chegam, entregando apenas o agregado final.

Por fim, uma verificação útil: o valor de `replies` deve ser igual ao número de comandos enviados. Uma divergência entre os dois pode indicar que o arquivo de entrada estava truncado ou malformado, mesmo que `errors` apareça como zero.

3.1.3 MongoDB

No SGBD MongoDB, o processo de importação de dados em formato CSV é realizado através do utilitário `mongoimport` ([MongoDB, Inc., 2026](#)). Sua sintaxe básica exige que se informe o banco de dados de destino através do parâmetro `--db`, seguido do nome da coleção que receberá os dados, especificado em `--collection`. Caso esse último parâmetro seja omitido, o nome da coleção será automaticamente inferido a partir do nome do arquivo CSV fornecido.

```
mongoimport --db nome_do_db --collection nome_da_colecao \  
--type csv --headerline --ignoreBlanks \  
--file caminho/do/arquivo.csv
```

O tipo do arquivo de entrada deve ser declarado explicitamente através do parâmetro `--type`, que neste caso recebe o valor `csv`. Para que o `mongoimport` reconheça os nomes dos campos a partir do próprio arquivo, utiliza-se o parâmetro `--headerline`, que instrui o utilitário a tratar o conteúdo da primeira linha do CSV como o cabeçalho dos campos. Opcionalmente, o parâmetro `--ignoreBlanks` pode ser utilizado para que valores em branco presentes no arquivo sejam ignorados durante a importação, evitando que campos vazios sejam persistidos na coleção. Por fim, o caminho do arquivo CSV a ser importado é fornecido através do parâmetro `--file`.

3.1.4 Cassandra

A importação de dados CSV para o Cassandra utiliza o comando `sstableloader` e envolve vários passos ([The Apache Software Foundation, 2026](#)). Inicialmente, é necessário se certificar que o arquivo CSV está no formato adequado para o Cassandra. Isso geralmente envolve garantir que os tipos de dados e as colunas correspondem ao esquema da tabela no Cassandra.

O `sstableloader` requer que os dados estejam em um formato específico chamado `SSTable`. Para converter o CSV em `SSTables`, você pode usar o comando `cqlsh` fornecido pelo Cassandra. Um exemplo de uso é o seguinte:

```
cqlsh -e "COPY keyspace_name.table_name TO 'output_directory';"
```

Na sequência, é possível usar o `sstableloader` para carregar os dados. A sintaxe do comando é a seguinte:

```
sstableloader -d <hostname> -u <username> -pw <password> \  
output_directory/keyspace_name/table_name
```

- `<hostname>`: O endereço do nó Cassandra.
- `<username>`: O nome de usuário, se a autenticação estiver habilitada.
- `<password>`: A senha correspondente.

Este comando move os dados do diretório de saída para o Cassandra.

3.1.5 Neo4j

Usar `neo4j-admin` para importar grandes conjuntos de dados no Neo4j envolve alguns passos (Neo4j, Inc., 2026). Esta ferramenta de linha de comando é projetada para importações em massa eficientes.

Certifique-se de que seus arquivos CSV estejam estruturados corretamente e sigam o formato necessário para nós e relacionamentos. Crie um banco de dados não povoado:

```
neo4j-admin create-db --database=nome_do_seu_banco_de_dados \
--from=diretorio_com_arquivos_csv
```

Substitua `nome_do_seu_banco_de_dados` pelo nome desejado para o novo banco de dados. A opção `--from` especifica o diretório onde estão localizados seus arquivos CSV. Execute o comando de importação com `neo4j-admin`. O comando exato depende do seu modelo de dados, mas pode se parecer com isto:

```
neo4j-admin import --database=nome_do_seu_banco_de_dados \
--mode=csv \
--nodes:Rotulo1 nos.csv \
--nodes:Rotulo2 nos2.csv \
--relationships:TIPO_RELACIONAMENTO relacionamentos.csv
```

Substitua *Rotulo1*, *Rotulo2*, *nos.csv*, *nos2.csv*, *TIPO_RELACIONAMENTO* e *relacionamentos.csv* pelos seus rótulos específicos e nomes de arquivo.

3.2 IMPORTAÇÃO DE ARQUIVO CSV PARA SGBD MULTIMODELO

Ferramentas comerciais e de código aberto tratam de **migração** ou **modelagem** entre tecnologias heterogêneas, embora raramente com o foco deste TCC (um único CSV largo + regras declarativas + cinco paradigmas).

- **ArangoDB** é um SGBD multimodelo nativo (documento/chave-valor/grafos) com `arangoimport` para CSV; a importação é para **um** motor multimodelo, não para cinco SGBDs independentes.
- `mongoimport` / `COPY` / `redis-cli --pipe` / `cqlsh` / `neo4j-admin` (seção 3.1) cobrem importação **para um único** produto por vez.
- **dbcrossbar** (DBCROSSBAR..., 2024) copia dados tabulares entre PostgreSQL, BigQuery, armazenamento em nuvem e CSV, incluindo conversão de esquemas; a ênfase é *pipeline* de migração, não mapeamento semântico de entidades aninhadas com filtros por valor.

- **Hackolade** ([Hackolade, 2024](#)) oferece *polyglot data modeling* com engenharia de esquema para dezenas de alvos; trata-se de **modelagem visual** e geração de artefatos, e não de um utilitário CLI único para importar um CSV operacional com validação prévia de filtros.

3.3 CONSIDERAÇÕES SOBRE OS TRABALHOS RELACIONADOS

A literatura sobre **metamodelos unificados** para SQL e NoSQL (por exemplo, U-Schema ([CANDEL; RUIZ; GARCÍA-MOLINA, 2022](#))) e sobre **evolução de esquema** em ambientes heterogêneos ([CHILLÓN et al., 2023](#); [CHILLÓN et al., 2024](#)) informa decisões de validação e de representação de entidades no JSON de configuração, mas não substitui uma ferramenta de *bulk import* orientada a CSV.

Surveys recentes convergem em desafios abertos da gestão poliglota de dados: migração automatizada entre *stores* quando o *workload* muda, planejamento de consultas entre sistemas, gestão de esquemas multi-modelo e preservação das propriedades não funcionais de cada SGBD ([KIEHN et al., 2022](#); [GLAKE et al., 2022](#)). [Roy-Hubara, Shoval e Sturm \(2022\)](#) tratam da **seleção de SGBDs** por fragmento de aplicação; [Silva, Bornia e Mello \(2024\)](#) mapeiam o estado da arte em **modelagem poliglota**. Nenhum desses trabalhos implementa, contudo, um utilitário declarativo que receba um CSV operacional único e o materialize simultaneamente em cinco paradigmas distintos com validação estática prévia.

Abordagens recentes reforçam essa tendência. [Pereira, Guerra e Rosa \(2025\)](#) propõem um *framework* de persistência poliglota **transparente**, com uma API unificada e extensível a diferentes tipos de banco, enquanto [Sachdeva e Vasava \(2024\)](#) exploram a combinação de bancos **multimodelo** e persistência poliglota para acomodar a variedade dos dados. Ambos os trabalhos atuam, porém, na **camada de acesso da aplicação** — oferecendo uma interface única para ler e gravar em múltiplos SGBDs em tempo de execução —, e não na **carga inicial declarativa** a partir de um CSV operacional, que é o foco da *Polyglot Import CSV*.

Sistemas *polystore* como o BigDAWG ([DUGGAN et al., 2015](#)) e *middlewares* de portabilidade em nuvem ([KAUR; SHARMA; KAHN, 2020](#)) tratam de **consultas federadas** e movimentação de dados em ecossistemas complexos. [Tan et al. \(2017\)](#) classificam essas soluções (federado, multistore, polystore, poliglota) e avaliam transparência, autonomia e heterogeneidade — eixos nos quais importadores nativos (seção 3.1) e ferramentas de migração (seção 3.2) também divergem, mas sem convergir para um *pipeline* único de ingestão.

Ferramentas como **dbcrossbar** ([DBCROSSBAR. . . , 2024](#)) e **Hackolade** ([Hackolade, 2024](#)) aproximam-se do domínio de integração de dados, porém com foco

em *pipelines* de migração ou modelagem visual, respectivamente — não em regras declarativas sobre entidades aninhadas, filtros por valor (*each*) e validação por JSON Schema antes de qualquer conexão com SGBDs.

Lacuna identificada: a combinação de (i) um CSV largo como fonte operacional, (ii) configuração JSON validada estaticamente, (iii) materialização distinta por paradigma (relacional, chave-valor, documento, colunar, grafo) e (iv) modo *dry-run* para revisão prévia. O *Polyglot Import CSV* ocupa essa lacuna como ferramenta de **bootstrap de persistência poliglota** — carga inicial e reproduzível de dados — enquanto *middlewares* e *polystores* assumem dados já residentes ou consultas em tempo de execução. Trabalhos futuros podem integrar a ferramenta a mecanismos de migração adaptativa descritos por [Kiehn et al. \(2022\)](#) e [Glake et al. \(2022\)](#).

4 A PROPOSTA DA FERRAMENTA POLYGLOTIMPORTCSV

4.1 VISÃO GERAL

A ferramenta **PolyglotImportCSV** (pacote Python `polyglot-import-csv`) importa os dados de um único arquivo CSV para vários SGBDs em uma única execução. Conforme ilustra a Figura 7, a ferramenta recebe **três entradas**: o arquivo de **dados** (CSV) e **dois arquivos de configuração** em formato JSON — o arquivo de **importação**, que mapeia as colunas do CSV para as entidades e relacionamentos desejados em cada SGBD, e o arquivo de **conexão**, que declara quais SGBDs estão disponíveis e como alcançá-los. O processamento é disparado por um comando de linha de comando (CLI), que primeiro **valida** as três entradas — cada arquivo de configuração é conferido contra um **JSON Schema** embutido e, em seguida, contra o próprio CSV — e só então executa a importação. A **saída** são os dados do CSV persistidos em cada SGBD presente na configuração, dentre os suportados: PostgreSQL, MongoDB, Cassandra, Redis e Neo4j.

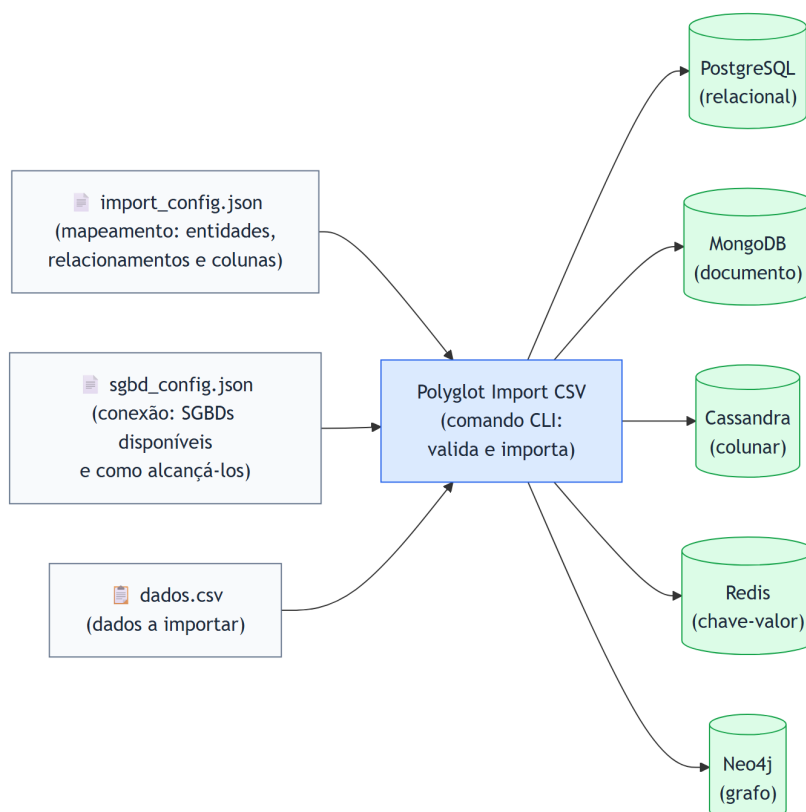


Figura 7 – Visão geral da ferramenta: entradas, processamento e saídas.

Fonte: Elaborado pelo autor (2026).

O formato dos dois arquivos de configuração é detalhado na seção 4.2; as

validações realizadas antes da importação, na seção 4.2.8; e o algoritmo de importação em si, na seção 4.3. O comando principal da CLI é o seguinte:

```
python -m polyglotimportcsv caminho/dados.csv \
  --config caminho/import_config.json \
  --sgbd-config caminho/sgbd_config.json \
  [--dry-run] \
  [--create-schema / --no-create-schema] \
  [--only postgres,redis]
```

Cada parte do comando tem o seguinte papel. O argumento `caminho/dados.csv` é obrigatório e indica o arquivo CSV com os dados a importar. A opção `--config` indica o caminho do arquivo JSON de importação (o mapeamento), e `--sgbd-config` indica o caminho do arquivo JSON de conexão dos SGBDs; se este último for omitido, a ferramenta procura um arquivo `sgbd_config.json` no mesmo diretório do arquivo de importação. As demais opções são facultativas: `--dry-run` executa somente as validações e reporta quantos registros **seriam** importados em cada destino, sem conectar a nenhum SGBD, o que é útil para revisar o mapeamento antes da carga real; `--create-schema` e `--no-create-schema` controlam se a ferramenta cria as estruturas de armazenamento necessárias (tabelas no PostgreSQL, *keyspace* e tabelas no Cassandra) antes de gravar os dados, sendo a criação ativada por padrão; e `--only` restringe a execução a um subconjunto dos SGBDs configurados (por exemplo, `--only postgres,redis`).

4.2 FORMATO DE CONFIGURAÇÃO (JSON SCHEMA)

A configuração é dividida em **dois arquivos JSON**, cada um validado estaticamente por um **JSON Schema** próprio embutido em `src/polyglotimportcsv/schemas/`: o `import_config.json` define o **mapeamento** de entidades, relacionamentos e colunas do CSV para cada SGBD; o `sgbd_config.json` define a **configuração de conexão** de cada SGBD, isto é, quais bancos estão disponíveis e como alcançá-los.

Essa separação atende a duas preocupações distintas: o *mapeamento* muda conforme o domínio dos dados; a *conexão* muda conforme o ambiente (desenvolvimento, testes, produção). Um SGBD só pode ser referenciado no arquivo de importação se estiver **declarado** no arquivo de conexão — esta e as demais regras de validação são detalhadas na seção 4.2.8. A representação completa de ambos os schemas na forma serializada JSON Schema 2020-12 encontra-se no Apêndice A.

4.2.1 Visão Geral da Estrutura dos Arquivos de Configuração

Antes de detalhar cada SGBD de destino suportado — identificado nos arquivos de configuração pelos blocos `postgres`, `mongodb`, `cassandra`, `redis` e `neo4j` — esta subseção apresenta a estrutura dos dois schemas como um todo, fornecendo um mapa

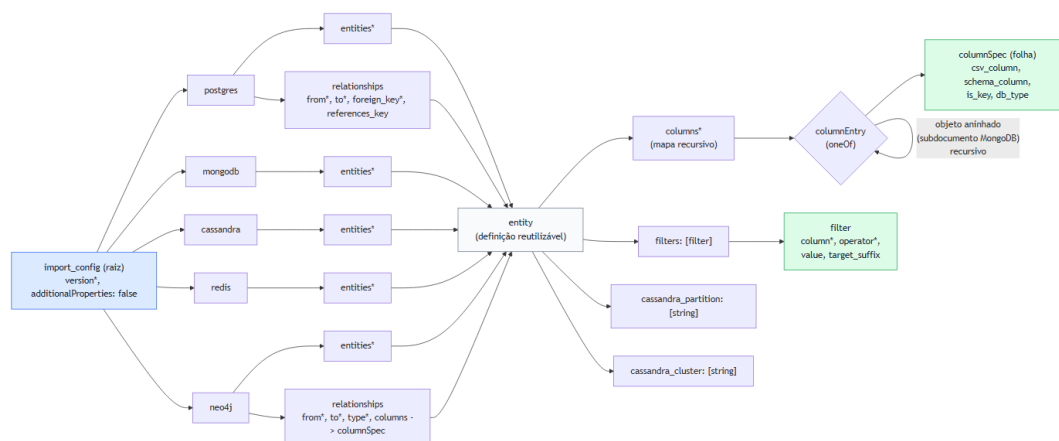


Figura 8 – Estrutura do JSON Schema de importação (`import_config`).

Fonte: Elaborado pelo autor (2026).

mental que orienta a leitura das subseções seguintes. A Figura 8 sintetiza graficamente o schema do arquivo de importação (`import_config`) e a Figura 9, o schema do arquivo de conexão (`sgbd_config`).

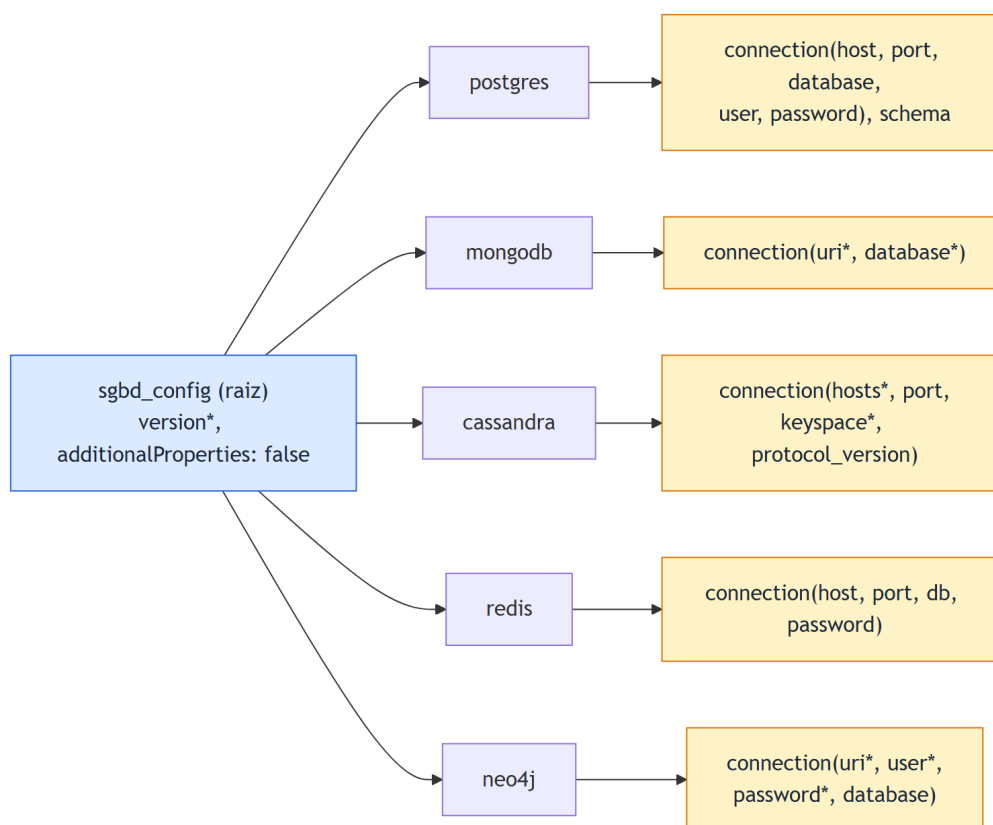


Figura 9 – Estrutura do JSON Schema de conexão (`sgbd_config`).

Fonte: Elaborado pelo autor (2026).

Raízes simétricas. Ambos os schemas partilham a mesma raiz: um campo `version` obrigatório e um bloco opcional por SGBD (`postgres`, `mongodb`, `cassandra`,

redis, neo4j), com `additionalProperties: false` rejeitando chaves desconhecidas. A diferença está no conteúdo de cada bloco: no arquivo de importação ele descreve o *que* mapear; no de conexão, *onde* gravar.

O arquivo de importação e a definição reutilizável `entity`. Seguindo as setas a partir de `import_config`, cada bloco de SGBD expande-se em um mapa `entities` (e, para PostgreSQL e Neo4j, também em `relationships`). O ponto central da Figura 8 — e da economia de vocabulário do schema — é que **todas as entidades, de todos os SGBDs, convergem para uma única definição reutilizável**, rotulada `entity`. Em vez de cinco formatos distintos, há apenas um: toda entidade possui um mapa `columns` (o mapeamento de colunas) e, opcionalmente, `filters` (predicados sobre as linhas do CSV) e os campos específicos do Cassandra (`cassandra_partition` e `cassandra_cluster`).

O mapeamento recursivo de colunas. À direita da Figura 8, `columns` remete a `columnEntry`, definido como uma escolha (`oneOf`) entre duas formas: (i) uma **folha** — o `columnSpec`, que liga uma coluna do CSV a um atributo do destino por meio dos campos `csv_column`, `schema_column`, `is_key` e `db_type`; ou (ii) um **objeto aninhado**, que contém novos `columnEntry`. Essa recursão é o que permite, com um único mecanismo, tanto o mapeamento plano (PostgreSQL, Cassandra, Redis) quanto os subdocumentos do MongoDB. As arestas tipadas do Neo4j e as chaves estrangeiras do PostgreSQL aparecem em `relationships`, reutilizando o mesmo `columnSpec` para eventuais propriedades.

O arquivo de conexão. Na Figura 9, cada bloco de SGBD de `sgbd_config` carrega apenas um descritor `connection` com os parâmetros de acesso pertinentes àquela tecnologia (por exemplo, `uri` para MongoDB e Neo4j; `hosts` e `keyspace` para Cassandra). É uma estrutura deliberadamente rasa: indica *onde* alcançar cada SGBD, sem nenhuma informação de mapeamento.

As subseções seguintes percorrem essa estrutura na ordem em que as duas figuras são lidas: primeiro a raiz e os elementos comuns a todos os SGBDs (`columnSpec`, `entity` e filtros) e, em seguida, as particularidades de cada SGBD, ilustradas com fragmentos concretos de configuração.

4.2.2 Raiz e Elementos Comuns

Como antecipado na visão geral, ambos os arquivos compartilham a mesma raiz: um campo `version` obrigatório (inteiro ≥ 1) e um bloco opcional por SGBD. A cláusula `additionalProperties: false` na raiz **rejeita** qualquer propriedade não prevista, impedindo que erros de digitação como `postgress` passem despercebidos.

`columnSpec`. Cada **folha** do mapa `columns` descreve como um valor do CSV alimenta um atributo no destino. Quatro campos opcionais compõem esse mapeamento,

conforme a Tabela 1.

Tabela 1 – Campos de mapeamento de coluna (columnSpec).

Campo	Tipo	Obrigatório	Descrição
csv_column	<i>string</i> ou inteiro (≥ 0)	não	Cabeçalho CSV ou índice da coluna (base 0). Padrão: chave JSON.
schema_column	<i>string</i>	não	Nome do atributo no SGBD destino. Padrão: chave JSON.
is_key	booleano	não	Chave primária, chave de MERGE (Neo4j) ou chave Redis.
db_type	<i>string</i>	não	Tipo SQL/CQL para geração de DDL.

Fonte: Elaborado pelo autor (2026).

A chave JSON de cada entrada em `columns` identifica o mapeamento e serve de **padrão** tanto para a coluna de origem no CSV quanto para o atributo de destino no SGBD; por isso a forma mínima (`"campo": {}`) basta quando os três nomes coincidem. Quando divergem, `csv_column` redefine a origem — por nome de cabeçalho ou por índice posicional de base 0, útil para CSVs sem cabeçalho — e `schema_column` redefine o destino no SGBD. Por ser um identificador independente da origem, a chave permite ainda que uma mesma coluna do CSV alimente atributos distintos (entradas com `csv_column` repetido), recurso pontual para cenários de denormalização.

entity. Uma entidade representa um alvo lógico: tabela (PostgreSQL/Cassandra), coleção (MongoDB), rótulo de nó (Neo4j) ou entidade Redis. Campos obrigatórios: `columns` (mapa recursivo de `columnSpec`). Opcionais: `filters` (predicados sobre o CSV), `cassandra_partition` e `cassandra_cluster` (apenas Cassandra).

Filtros. Cada item exige `column` e `operator` (`==`, `!=`, `>`, `<`, `>=`, `<=`, `in`, `not_in`, `each`). O operador `each` particiona a entidade por valores distintos da coluna, gerando múltiplos alvos com sufixo opcional.

4.2.3 PostgreSQL

Arquivo de importação. O bloco `postgres` contém `entities` (entidades planas — aninhamento de `columns` é rejeitado) e, opcionalmente, `relationships` para declarar chaves estrangeiras. O campo `db_type` em cada `columnSpec` alimenta a geração de

DDL (CREATE TABLE) quando --create-schema está ativo. Um exemplo é mostrado no Listagem 4.1, referente à importação de dados no domínio de e-commerce.

```
"postgres": {
  "entities": {
    "inventory": {
      "columns": {
        "product_id": { "is_key": true, "db_type": "BIGINT" },
        "quantity_available": { "db_type": "BIGINT" },
        "last_restock_date": { "db_type": "TIMESTAMPZ" },
        "price": { "db_type": "NUMERIC(14,2)" }
      },
      "filters": [{ "column": "action", "operator": "=", "value": "stock" }]
    },
    ...
  },
  "relationships": {
    "product_category": {
      "from": "products",
      "to": "categories",
      "foreign_key": "category_id",
      "references_key": "category_id"
    },
    ...
  }
}
```

Listagem 4.1 – Fragmento de importação PostgreSQL (e-commerce).

As reticências indicam que o arquivo real declara outras três entidades (categories, products e orders) e uma segunda chave estrangeira (order_product), omitidas aqui por brevidade; o arquivo completo é reproduzido no Apêndice C. O campo filters restringe a entidade inventory às linhas do CSV cujo valor da coluna action seja stock; a sintaxe geral de filtros foi apresentada anteriormente, em **Filtros**.

Arquivo de conexão. O bloco postgres contém connection (host, port, database, user, password) e o campo schema (padrão public), que define o schema PostgreSQL de destino das tabelas. Um exemplo de configuração de conexão é mostrado na Listagem 4.2.

```
"postgres": {
  "connection": {
    "host": "127.0.0.1", "port": 5432,
    "database": "ecommerce",
    "user": "postgres", "password": "postgres"
  },
  "schema": "public"
}
```

Listagem 4.2 – Fragmento de conexão PostgreSQL.

4.2.4 MongoDB

Arquivo de importação. O bloco `mongodb` contém apenas `entities`. O diferencial do MongoDB é o suporte a **aninhamento de columns**: chaves JSON aninhadas produzem subdocumentos BSON, sem necessidade de nenhum campo adicional — o próprio mapa recursivo expressa a hierarquia. A Listagem 4.3 exemplifica esse aninhamento: as chaves `category` e `stock` não são colunas do CSV, e sim objetos que agrupam outras colunas, produzindo dois subdocumentos dentro de cada documento `product_catalog`.

```
"mongodb": {
  "entities": {
    "product_catalog": {
      "columns": {
        "product_id": {},
        "product_name": {},
        ...
        "category": {
          "category_id": {},
          "category_name": {}
        },
        "stock": {
          "quantity_available": {},
          "last_restock_date": {}
        }
      },
      "filters": [{ "column": "action", "operator": "=", "value": "stock" }]
    }
  }
}
```

Listagem 4.3 – Fragmento de importação MongoDB com subdocumentos.

As reticências omitem as demais colunas cadastrais do produto (variante, marca, descrição, imagem e preço), mapeadas de forma direta como `product_id` e `product_name`; o bloco completo encontra-se no Apêndice C.

Arquivo de conexão. O bloco `mongodb` contém `connection` com `uri` (string de conexão MongoDB, incluindo usuário e senha quando necessário) e `database` (nome do banco de destino), como no Listagem 4.4.

```
"mongodb": {
  "connection": {
    "uri": "mongodb://127.0.0.1:27017",
    "database": "ecommerce"
  }
}
```

Listagem 4.4 – Fragmento de conexão MongoDB.

4.2.5 Apache Cassandra

Arquivo de importação. O bloco `cassandra` contém `entities` planas. Dois campos específicos controlam a chave primária composta do Cassandra: `cassandra_partition` (lista de colunas CSV que formam a chave de partição) e `cassandra_cluster` (colunas de agrupamento). O campo `cassandra_partition` é obrigatório para gerar o schema Cassandra: sua ausência só é detectada quando `--create-schema` está ativo, pois o CQL exige ao menos uma coluna de partição na cláusula `PRIMARY KEY`; esse campo é independente de `is_key`, que no Cassandra não tem papel na definição da chave primária. A Listagem 4.5 ilustra uma chave composta.

```
"cassandra": {
  "entities": {
    "user_activity_log": {
      "columns": {
        "user_id": {},
        "timestamp": { "schema_column": "event_time" },
        "action": { "schema_column": "event_type" },
        "product_id": {},
        "order_number": {},
        "selected_product_id": {},
        "shopping_cart_id": {}
      },
      "cassandra_partition": ["user_id"],
      "cassandra_cluster": ["timestamp"]
    }
  }
}
```

Listagem 4.5 – Fragmento de importação Cassandra com chave composta.

O campo `schema_column` em `timestamp` e em `action` renomeia essas colunas do CSV para `event_time` e `event_type` na tabela CQL — exemplo concreto de como `csv_column` e `schema_column` atuam independentemente.

Arquivo de conexão. O bloco `cassandra` contém `connection` com `hosts` (lista de endereços de nós), `port` (padrão 9042), `keyspace` e `protocol_version` opcional, conforme o Listagem 4.6.

```
"cassandra": {
  "connection": {
    "hosts": ["localhost"],
    "port": 9042,
    "keyspace": "ecommerce"
  }
}
```

Listagem 4.6 – Fragmento de conexão Cassandra.

4.2.6 Redis

Arquivo de importação. O bloco redis contém entities. A restrição fundamental é que **cada entidade deve ter exatamente uma coluna marcada com is_key**: esse valor se torna a chave Redis — usada exatamente como aparece no CSV, sem prefixo de entidade — e todos os demais campos da linha compõem o valor JSON armazenado. A Listagem 4.7 mostra as duas entidades do cenário e-commerce.

```
"redis": {
  "entities": {
    "shopping_cart": {
      "columns": {
        "shopping_cart_id": { "is_key": true },
        "cart_product_id": {},
        "cart_quantity": {}
      },
      "filters": [{ "column": "action", "operator": "==", "value": "add_to_cart" }]
    },
    "user_session": {
      "columns": {
        "user_id": { "is_key": true },
        "user_name": {},
        "user_email": {},
        "timestamp": { "schema_column": "last_seen" }
      },
      "filters": [{ "column": "action", "operator": "==", "value": "select_product" }]
    }
  }
}
```

Listagem 4.7 – Fragmento de importação Redis (e-commerce).

A entidade shopping_cart usa como chave o identificador do carrinho (shopping_cart_id) e restringe-se às linhas de adição ao carrinho; a entidade user_session usa como chave o identificador do usuário e renomeia timestamp para last_seen, registrando a última atividade de navegação de cada usuário.

Arquivo de conexão. O bloco redis contém connection com host, port (padrão 6379), db (índice do banco, padrão 0) e password opcional, como na Listagem 4.8.

```
"redis": {
  "connection": {
    "host": "127.0.0.1",
    "port": 6379,
    "db": 0
  }
}
```

Listagem 4.8 – Fragmento de conexão Redis.

4.2.7 Neo4j

Arquivo de importação. O bloco neo4j contém entities (nós) e relationships (arestas tipadas). Cada entidade declara um rótulo (*label*) de nó e deve ter exatamente uma coluna is_key, usada como predicado de identificação no MERGE Cypher. As arestas em relationships declaram from (rótulo de origem), to (rótulo de destino), type (tipo da aresta em maiúsculas) e um mapa columns opcional com propriedades da aresta. A Listagem 4.9 mostra um nó de cada rótulo e uma aresta entre eles.

```
"neo4j": {
  "entities": {
    "User": {
      "columns": {
        "user_id": { "is_key": true },
        "user_name": {},
        "user_email": {}
      },
      "filters": [{ "column": "action", "operator": "=", "value": "purchase" }]
    },
    "Product": {
      "columns": {
        "product_id": { "is_key": true },
        "product_name": {},
        "product_brand": {}
      },
      "filters": [{ "column": "action", "operator": "=", "value": "stock" }]
    }
  },
  "relationships": {
    "PURCHASED": {
      "from": "User",
      "to": "Product",
      "type": "PURCHASED",
      "columns": {
        "order_number": { "is_key": true },
        "quantity": {},
        "price": {},
        "rating": {}
      }
    }
  }
}
```

Listagem 4.9 – Fragmento de importação Neo4j (nós e aresta).

Arquivo de conexão. O bloco neo4j contém connection com uri (bolt://...), user, password e database opcional (padrão neo4j), conforme o Listagem 4.10.

```
"neo4j": {
  "connection": {
    "uri": "bolt://127.0.0.1:7687",
    "user": "neo4j",
    "password": "password",
    "database": "neo4j"
  }
}
```

```
}  
}
```

Listagem 4.10 – Fragmento de conexão Neo4j.

4.2.8 Validação, Consistência entre Arquivos e Fluxo de Carregamento

A validação ocorre em **três camadas** antes de qualquer conexão com os SGBDs. A primeira é a validação por **JSON Schema**: cada arquivo é validado contra o seu schema (sintaxe, tipos e propriedades permitidas) — `import_config.schema.json` para o arquivo de importação e `sgbd_config.schema.json` para o arquivo de conexão. A segunda é a **consistência entre arquivos** (`config_parser.merge_configs`): todo SGBD referenciado no arquivo de importação deve estar declarado no arquivo de conexão. A terceira é a **validação cruzada com o CSV** (`validation.validate_import_config`), que confere se as colunas referenciadas existem no CSV, se os filtros são coerentes, se as chaves estrangeiras do PostgreSQL e as arestas do Neo4j referenciam entidades válidas, se os SGBDs de modelo plano não usam `columns` aninhados e se os índices `csv_column` estão dentro do intervalo válido.

Qualquer falha levanta `BusinessException` e **impede** o início da importação. A Figura 10 apresenta o fluxo completo de carregamento, com os pontos de falha de cada camada e a convergência dos dois arquivos em um único `runtime_config`, ao qual o CSV se junta apenas na validação semântica (camada 3). O diagrama evidencia por que a separação em dois arquivos isola falhas: um `sgbd_config.json` inválido é detectado independentemente do `import_config.json`, sem necessidade de ler o CSV.

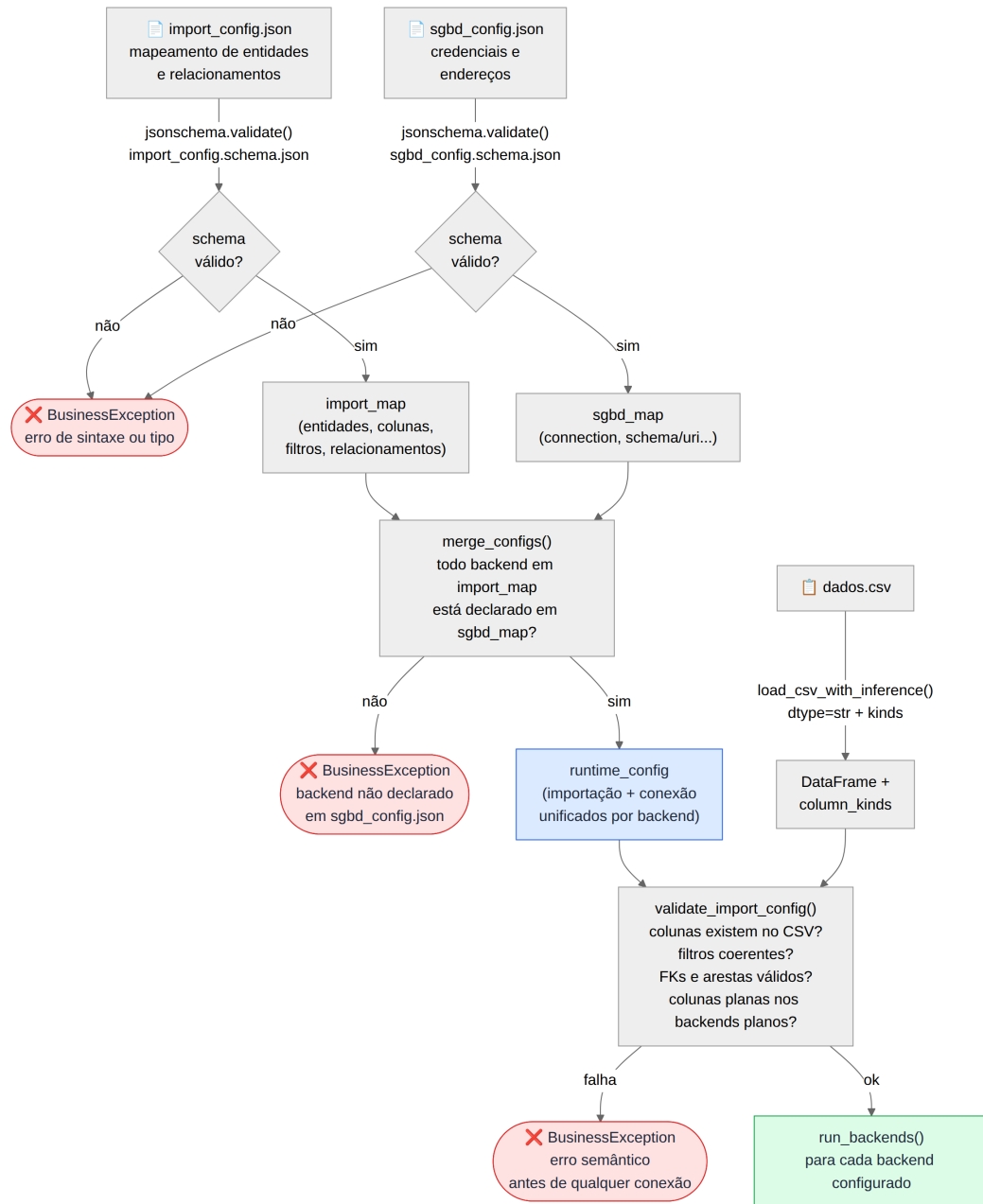


Figura 10 – Fluxo de carregamento e validação dos dois arquivos de configuração.

Fonte: Elaborado pelo autor (2026).

4.3 ALGORITMO DE EXECUÇÃO DA IMPORTAÇÃO

Definidos o formato de configuração e as validações, esta seção descreve como a ferramenta executa a importação propriamente dita, desde o comando de linha de comando apresentado na seção 4.1 até a persistência em cada SGBD. A Figura 11 apresenta o fluxo completo de execução; cada etapa do diagrama é explicada textualmente na sequência.

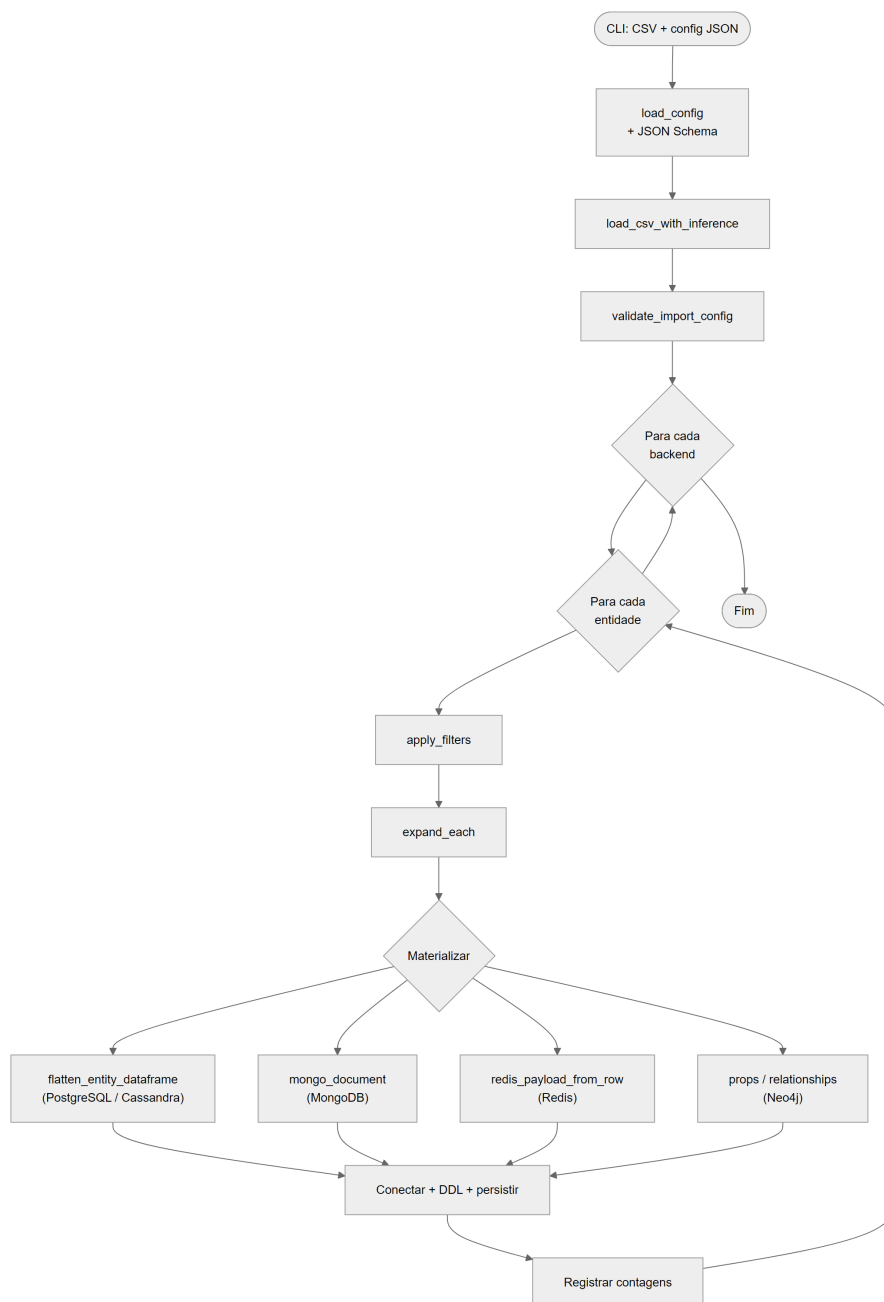


Figura 11 – Algoritmo de execução da importação Polyglot Import CSV.

Fonte: Elaborado pelo autor (2026).

A execução parte da **entrada** fornecida pelo usuário na linha de comando: o caminho do arquivo CSV, os caminhos dos dois arquivos JSON de configuração (importação e conexão) e as opções de execução `--dry-run`, `--create-schema/--no-create-schema` e `--only`, descritas na seção 4.1.

Na etapa de **carregamento da configuração**, os dois arquivos JSON são lidos e validados contra os seus respectivos JSON Schemas (camada 1 da seção 4.2.8); em seguida, verifica-se a consistência entre os arquivos — todo SGBD referenciado no arquivo de importação deve estar declarado no arquivo de conexão (camada 2) — e as

conexões são mescladas em uma configuração única de execução.

Na etapa de **carregamento do CSV**, todos os valores são lidos inicialmente como texto, preservando o conteúdo original do arquivo e evitando conversões prematuras de tipo (por exemplo, em datas com fuso horário). Em seguida, a ferramenta classifica cada coluna segundo o tipo de dado predominante entre os seus valores — inteiro, real, data/hora, texto ou vazia. Essa classificação alimenta a etapa seguinte, a **validação cruzada entre configuração e CSV** (camada 3 da seção 4.2.8). Qualquer falha, em qualquer camada, encerra a execução com uma mensagem de erro antes que qualquer conexão com os SGBDs seja aberta.

Validadas as entradas, inicia-se o núcleo do algoritmo, que percorre os SGBDs presentes na configuração (excluídos os não selecionados por `--only`) e, para cada um, as entidades declaradas em `entities`. O tratamento de cada entidade ocorre em duas fases: primeiro delimita-se o subconjunto de linhas do CSV que pertence à entidade; depois, cada linha desse subconjunto é transformada na estrutura de dados própria do modelo de destino.

Na primeira fase, a ferramenta **aplica os filtros** declarados na configuração, operação que corresponde à *seleção* da álgebra relacional: apenas as linhas que satisfazem os predicados (por exemplo, `action == stock`) participam da entidade. Havendo filtro com o operador `each`, a entidade é adicionalmente **particionada** pelos valores distintos da coluna indicada — um particionamento horizontal em que cada valor dá origem a um destino próprio, identificado por sufixo.

Na segunda fase, a **materialização**, cada linha selecionada é convertida na estrutura do modelo de dados de destino, segundo os conceitos revisados no capítulo 2. Para os SGBDs de organização tabular (o relacional PostgreSQL e o colunar Cassandra), a conversão combina três operações também descritas pela álgebra relacional: *projeção* (somente as colunas mapeadas em `columns` são mantidas), *renomeação* (aplicação de `schema_column`) e *eliminação de duplicatas* com base nas colunas de chave (`is_key`), de modo que cada tabela receba apenas tuplas únicas. Para o modelo de documentos (MongoDB), os valores da linha são agregados em um documento cuja hierarquia reproduz o aninhamento declarado em `columns`, materializando a representação desnormalizada característica desse modelo. Para o modelo chave-valor (Redis), cada linha dá origem a um par em que a chave é o valor da coluna marcada com `is_key` e o valor é a representação textual (JSON) dos demais campos mapeados. Para o modelo de grafo (Neo4j), cada linha dá origem a um nó com as propriedades mapeadas; na sequência, as arestas declaradas em `relationships` conectam os nós criados, com semântica de mesclagem — a aresta só é criada se ainda não existir, evitando duplicação quando a mesma relação aparece em várias linhas do CSV.

Por fim, para cada entidade materializada, a ferramenta **conecta-se** ao SGBD (etapa suprimida quando `--dry-run` está ativo), **cria as estruturas de armazenamento** quando `--create-schema` está ativo (tabelas no PostgreSQL; *keyspace* e tabelas no Cassandra) e **grava** os registros por meio da operação de escrita própria de cada tecnologia. A **saída** da execução é um relatório com as contagens de registros gravados por entidade e por SGBD, que permite conferir o resultado da carga.

4.3.1 O Cenário de Exemplo e o Arquivo de Dados

Para concretizar o algoritmo, esta subseção e a seguinte percorrem o cenário de e-commerce que acompanha o repositório da ferramenta — o mesmo cenário cujos fragmentos de configuração ilustraram a seção 4.2 e cuja execução completa é demonstrada na seção 4.3.3.

O arquivo de dados do cenário, `ecommerce_join.csv`, registra 32 eventos de uma loja virtual em 39 colunas. O arquivo resulta da **junção** (*join*) de todas as coleções de dados da fonte — usuários, endereços, produtos, categorias, estoque, pedidos, seleções de produto e carrinhos de compra — em uma única tabela larga, como indica o próprio sufixo `_join` do nome. Por isso ele é **esparso**: cada linha representa um evento, identificado pela coluna `action` (`stock`, `purchase`, `select_product` ou `add_to_cart`), e preenche apenas o subconjunto de colunas pertinente àquele tipo de evento, permanecendo vazias as demais. O Apêndice B reproduz o arquivo completo em forma tabular, organizado por ação, e indica quais grupos de colunas cada ação preenche.

A Tabela 2 apresenta as oito linhas de reposição de estoque (`action == stock`), que servem de fio condutor do exemplo. Para caber na página, a tabela resume as colunas de identificação, cujos valores seguem padrões regulares no arquivo: o produto `ProductN` tem `product_id N` e pertence à categoria `CategoriaN` (`category_id N`); o usuário `userN` é identificado por um `user_id` alfanumérico (por exemplo, `7fd7eee7-...` para `user1`), tem e-mail `userN@example.com` e um endereço próprio, detalhado no Apêndice B; a descrição e a imagem do produto seguem padrões análogos. Todos os eventos de estoque ocorrem em 02/11/2023, e a coluna `last_restock_date` repete o instante do próprio evento.

Tabela 2 – Linhas `action == stock` do `ecommerce_join.csv` (colunas de identificação resumidas).

Hora	Usuário	Produto	Variante	Marca	Categoria	Qtd.	Preço
03:30	user1	Product2	blue	Microsoft	Categoria2	2	50.5
06:45	user2	Product3	green	Google	Categoria3	1	52.07
09:15	user2	Product9	black	Pepsi	Categoria9	67	32.78
12:00	user8	Product8	white	Google	Categoria8	60	46.79
14:30	user9	Product4	yellow	Apple	Categoria4	82	55.94
17:15	user9	Product5	red	Samsung	Categoria5	57	34.21
20:00	user9	Product6	orange	Coca-Cola	Categoria6	64	51.98
22:30	user9	Product7	grey	Audi	Categoria7	1	100000

Fonte: Elaborado pelo autor (2026), a partir de `data/ecommerce/ecommerce_join.csv`.

4.3.2 A Importação em Cada SGBD

Apresentado o arquivo de dados, esta subseção acompanha o percurso dessas linhas pelo núcleo do algoritmo, SGBD a SGBD. Em cada caso estão em jogo as três entradas da ferramenta: as linhas do CSV (Tabela 2), o trecho correspondente do arquivo de importação e o arquivo de conexão — trechos reproduzidos nas listagens da seção 4.2 e, na íntegra, no Apêndice C. O que varia de um SGBD para outro é a estrutura de dados produzida como saída, apresentada a seguir.

PostgreSQL. O arquivo de importação declara quatro entidades (Listagem 4.1). A entidade `inventory` seleciona as linhas de estoque (`action == stock`) e projeta quatro colunas, com o `product_id` como chave: o resultado é a relação da Tabela 3, criada e gravada no banco `ecommerce` declarado no arquivo de conexão (Listagem 4.2). As entidades `categories` e `products` não declaram filtro — a seleção abrange o arquivo inteiro —, mas a eliminação de duplicatas pela chave (e o descarte das linhas em que a chave está vazia, caso das ações de navegação) reduz o resultado aos oito produtos e oito categorias distintos; a entidade `orders` seleciona as oito linhas de compra. As chaves estrangeiras declaradas em `relationships` ligam produtos a categorias e pedidos a produtos.

Tabela 3 – Relação inventory materializada no PostgreSQL a partir das linhas de estoque.

product_id	quantity_available	last_restock_date	price
2	2	02/11/2023 03:30	50.5
3	1	02/11/2023 06:45	52.07
9	67	02/11/2023 09:15	32.78
8	60	02/11/2023 12:00	46.79
4	82	02/11/2023 14:30	55.94
5	57	02/11/2023 17:15	34.21
6	64	02/11/2023 20:00	51.98
7	1	02/11/2023 22:30	100000

Fonte: Elaborado pelo autor (2026).

MongoDB. A entidade `product_catalog` (Listagem 4.3) também seleciona as linhas de estoque, mas as materializa como documentos: cada linha da Tabela 2 torna-se um documento cuja hierarquia reproduz o aninhamento declarado em `columns`. A Listagem 4.11 mostra o documento gerado a partir da primeira linha de estoque. Os valores preservam a forma textual lida do CSV, coerente com a política de carregamento descrita no início desta seção; os subdocumentos `category` e `stock` agrupam, sem repetição de linhas, o que no PostgreSQL ficou distribuído em três tabelas ligadas por chaves estrangeiras — a representação desnormalizada característica do modelo de documentos.

```
{
  "product_id": "2",
  "product_name": "Product2",
  "product_variant": "blue",
  "product_brand": "Microsoft",
  "product_description": "Description1",
  "product_image": "https://example.com/image1.jpg",
  "price": "50.5",
  "category": {
    "category_id": "2",
    "category_name": "Categoria2"
  },
  "stock": {
    "quantity_available": "2",
    "last_restock_date": "2023-11-02 03:30:00Z"
  }
}
```

Listagem 4.11 – Documento gerado na coleção `product_catalog` para a primeira linha de estoque.

Cassandra. A entidade `user_activity_log` (Listagem 4.5) não declara filtro: todos os 32 eventos do arquivo alimentam a tabela, com `timestamp` e `action` renomeados para `event_time` e `event_type`. A chave primária composta — partição por `user_id`, agrupamento por `timestamp` — faz com que todos os eventos de um mesmo usuário fiquem na mesma partição, ordenados no tempo. A Tabela 4 mostra a partição do usuário `user1`: os seus quatro eventos, um de cada tipo, com as colunas de referência preenchidas conforme a ação — a mesma esparsidade do CSV de origem, agora a serviço de uma consulta típica do modelo colunar (“o histórico de atividades de um usuário”).

Tabela 4 – Partição do usuário `user1` na tabela `user_activity_log` (Cassandra).

<i>event_time</i>	<i>event_type</i>	<i>product_id</i>	<i>order_number</i>	<i>selected_product_id</i>	<i>shopping_cart_id</i>
02/11 03:30	stock	2	—	—	—
15/11 15:09	select_product	—	—	11	—
15/11 15:17	add_to_cart	—	—	—	user:7fd7...:cart
15/11 15:46	purchase	2	ORD1	—	—

Fonte: Elaborado pelo autor (2026).

Redis. As linhas de estoque não alimentam o Redis: as duas entidades desse SGBD (Listagem 4.7) selecionam as ações de navegação — `add_to_cart` para `shopping_cart` e `select_product` para `user_session`. Cada linha selecionada torna-se um par chave-valor: a chave é o valor da coluna marcada com `is_key` e o valor é a representação textual (JSON) dos demais campos mapeados. A Listagem 4.12 mostra os dois pares gerados a partir dos eventos de `user1`; ao todo, são oito carrinhos e oito sessões. Note-se que o padrão `user:...:cart` da chave do carrinho vem do próprio CSV (coluna `shopping_cart_id`), e não de alguma convenção da ferramenta.

```

chave: user:7fd7eee7-4675-4f56-ad81-108da6cecebc:cart
valor: {"cart_product_id": "3", "cart_quantity": "1"}
chave: 7fd7eee7-4675-4f56-ad81-108da6cecebc
valor: {"user_name": "user1", "user_email": "user1@example.com",
      "last_seen": "2023-11-15 15:09:30Z"}

```

Listagem 4.12 – Pares chave-valor gerados no Redis a partir dos eventos de `user1`.

Neo4j. As entidades e a aresta da Listagem 4.9 produzem um grafo de propriedades: as oito linhas de estoque dão origem aos nós `Product`; as oito linhas de compra

dão origem aos nós User — repare que user9, que apenas vende e nunca compra, não gera nó — e às oito arestas PURCHASED, identificadas pelo número do pedido e decoradas com quantidade, preço e nota da avaliação. A semântica de mesclagem garante que nós e arestas não se dupliquem quando a mesma chave reaparece. A Listagem 4.13 representa, na notação usual de grafos de propriedades, o fragmento do grafo derivado da primeira compra (ORD1).

```
(:User {user_id: "7fd7eee7-...", user_name: "user1",
      user_email: "user1@example.com"})
  -[:PURCHASED {order_number: "ORD1", quantity: "1",
               price: "50.5", rating: "2"}]->
(:Product {product_id: "2", product_name: "Product2",
          product_brand: "Microsoft"})
```

Listagem 4.13 – Fragmento do grafo materializado no Neo4j para o pedido ORD1.

A Tabela 5 consolida o percurso, tomando por referência a primeira linha de dados da Tabela 2 (o produto Product2): cada célula da coluna “Saída materializada” decorre diretamente das entidades, filtros e columns declarados no arquivo de importação.

Tabela 5 – Materialização de uma linha action == stock nos diferentes SGBDs.

SGBD	Entidade	Filtro	Saída materializada
PostgreSQL	inventory (a linha também alimenta products e categories)	action == stock	Linha da Tabela 3: product_id, quantity_available, last_restock_date, price.
MongoDB	product_catalog	action == stock	Documento da Listagem 4.11, com subdocumentos category e stock aninhados.
Cassandra	user_activity_log	nenhum (todas as linhas)	Registro com timestamp e action renomeados para event_time e event_type; chave primária composta (user_id, timestamp).
Redis	—	—	Entidades Redis selecionam outras ações; esta linha não alimenta o Redis.
Neo4j	nó Product	action == stock	Nó :Product da Listagem 4.13, identificado por product_id.

Fonte: Elaborado pelo autor (2026).

Assim, um **único CSV largo** alimenta destinos heterogêneos: tabelas relacionais filtradas por tipo de evento, documentos aninhados, *log* colunar com renomeação de colunas, pares chave-valor e um grafo de propriedades — materializando a persistência poliglota descrita no capítulo 2. As contagens observadas neste percurso (oito registros por entidade filtrada e 32 no registro de atividades) reaparecem na verificação prévia da execução, apresentada a seguir.

4.3.3 Exemplo de Uso e Evidência de Execução (Cenário E-commerce)

Esta seção demonstra o uso da ferramenta em uma execução completa do cenário de e-commerce que acompanha o código-fonte. Todos os arquivos de entrada encontram-se no diretório `data/ecommerce/` do repositório da ferramenta: o arquivo de dados `ecommerce_join.csv`, com 32 linhas de eventos de uma loja virtual (ações `stock`, `purchase`, `add_to_cart` e `select_product`), reproduzido em forma tabular no Apêndice B; o arquivo de importação `import_config.json`, cujos fragmentos por SGBD foram apresentados nas Listagens 4.1 a 4.9; e o arquivo de conexão `sgbd_config.json`, cujos fragmentos acompanham as mesmas subseções — ambos reproduzidos na íntegra no Apêndice C. Além deles, o arquivo `docker-compose.yml`, na raiz do repositório, define os contêineres dos cinco SGBDs usados no exemplo. A execução usa o comando apresentado na seção 4.1, instanciado com esses arquivos:

```
python -m polyglotimportcsv data/ecommerce/ecommerce_join.csv \
  --config data/ecommerce/import_config.json \
  --sgbd-config data/ecommerce/sgbd_config.json \
  --dry-run
```

Com a *flag* `--dry-run`, a ferramenta executa todas as validações e a materialização, mas não abre conexão com nenhum SGBD — o *driver* Apache Cassandra, por exemplo, sequer é importado, o que permite o *dry-run* em ambientes onde a sua extensão C não está disponível. O resultado são as contagens de registros por destino mostradas na Tabela 6, úteis para revisar o mapeamento antes da carga real.

Tabela 6 – Contagens reportadas pelo modo `--dry-run` no cenário e-commerce.

SGBD	Destino	Registros (após filtros/deduplicação)
PostgreSQL	categories	8
PostgreSQL	products	8
PostgreSQL	inventory	8
PostgreSQL	orders	8
MongoDB	product_catalog	8 documentos
Cassandra	user_activity_log	32 linhas (todas as ações)
Redis	shopping_cart	8
Redis	user_session	8
Neo4j	nó User	8
Neo4j	nó Product	8
Neo4j	aresta PURCHASED	8 (uma por linha purchase)

Fonte: Elaborado pelo autor (2026).

Para a importação real, basta subir os contêineres com `docker compose up -d` e repetir o mesmo comando sem a *flag* `--dry-run`: os mesmos volumes da Tabela 6 são então persistidos nos cinco SGBDs configurados, materializando o cenário poliglota da seção 2.1. Alternativamente, o script `run_example.sh`, na raiz do repositório, automatiza o fluxo completo: lê o `sghbd_config.json`, sobe apenas os contêineres dos SGBDs ali declarados e, em seguida, executa o *dry-run* e a importação real com `--create-schema`.

Complementarmente à evidência de execução, testes automatizados (pytest) cobrem a validação por schema, a separação e mesclagem dos dois arquivos de configuração, o mapeamento de colunas (`csv_column`, `schema_column`, `aninhamento`), os filtros e *smoke tests* de validação com *dry-run*.

5 ATIVIDADES FUTURAS

Como continuidade deste trabalho, as seguintes atividades estão previstas para o TCC II, abrangendo tanto a avaliação da ferramenta quanto a ampliação de suas funcionalidades:

- **Avaliação de desempenho** das importações com diferentes volumes de dados CSV e configurações, comparando importações simples e complexas para os diferentes modelos de dados destino.
- **Interface gráfica *desktop*** (semestre seguinte) que monte o comando CLI e visualize os arquivos JSON de configuração.
- **Suporte a TSV/Excel** e a processamento *chunked* para CSV maiores que a RAM.
- **Importação com um arquivo CSV por entidade.** Identificar a origem de cada dado é um requisito essencial do processo de importação: no formato atual, cada linha do CSV precisa indicar a entidade a que pertence, papel cumprido pela coluna discriminadora *action* em conjunto com os *filters* da configuração (Seção 4.2). Uma evolução prevista é aceitar um arquivo CSV por entidade, em que o próprio arquivo designa a origem dos dados, dispensando a coluna discriminadora e os filtros correspondentes; o CSV único combinado permaneceria como modo alternativo de entrada.
- **Operadores de filtro adicionais** e políticas de coerção de tipos configuráveis.
- **Testes de integração** contra *docker compose* em CI.
- **Generalização** do núcleo de importação para novos conectores (mensageria, *data lakes*, outros SGBDs).

REFERÊNCIAS

- CANDEL, C. J. F.; RUIZ, D. S.; GARCÍA-MOLINA, J. J. A unified metamodel for NoSQL and relational databases. *Information Systems*, v. 104, p. 101898, 2022. Citado na página 26.
- CHILLÓN, A. H. et al. A generic schema evolution approach for nosql and relational databases. *IEEE Transactions on Knowledge and Data Engineering*, v. 36, n. 7, p. 2774–2789, 2024. Citado na página 26.
- CHILLÓN, A. H. et al. Propagating schema changes to code: An approach based on a unified data model. In: *Proceedings of the Workshops of the EDBT/ICDT 2023 Joint Conference (CoMoNoS)*. [s.n.], 2023. (CEUR Workshop Proceedings, v. 3379). Disponível em: <https://ceur-ws.org/Vol-3379/CoMoNoS_2023_id251_Alberto_Hernandez_Chillon.pdf>. Citado 2 vezes nas páginas 11 e 26.
- DBCROSSBAR: copy tabular data between databases, CSV files, and cloud storage. 2024. <<https://www.dbcrossbar.org>>. Ferramenta de código aberto em Rust para migração e conversão de esquemas. Citado 2 vezes nas páginas 25 e 26.
- DUGGAN, J. et al. The BigDAWG polystore system. *ACM SIGMOD Record*, v. 44, n. 2, p. 11–16, 2015. Citado na página 26.
- FORD, N. *The Productive Programmer*. [S.l.]: O'Reilly Media, 2008. Citado na página 14.
- FOWLER, M. *PolyglotPersistence*. 2011. Disponível em: <<https://martinfowler.com/bliki/PolyglotPersistence.html>>. Citado na página 14.
- GLAKE, D. et al. *Towards Polyglot Data Stores: Overview and Open Research Questions*. 2022. Citado 3 vezes nas páginas 20, 26 e 27.
- Hackolade. *Polyglot Data Modeling*. 2024. <<https://hackolade.com/polyglot-data-modeling.html>>. Modelagem conceitual com engenharia de esquema para múltiplos SGBDs. Citado na página 26.
- HECHT, R.; JABLONSKI, S. Nosql evaluation: A use case oriented survey. In: *Proceedings of the 2011 International Conference on Cloud and Service Computing*. [S.l.: s.n.], 2011. p. 336–341. Citado na página 16.
- KAUR, K.; SHARMA, S.; KAHN, K. S. A middleware for polyglot persistence and data portability of big data PaaS cloud applications. *Computers, Materials & Continua*, v. 65, n. 2, p. 1625–1647, 2020. Citado na página 26.
- KHINE, P. P.; WANG, Z. A review of polyglot persistence in the big data world. *Information*, v. 10, n. 4, p. 141, 2019. Citado na página 11.
- KIEHN, F. et al. Polyglot data management: State of the art & open challenges. *Proceedings of the VLDB Endowment*, v. 15, n. 12, p. 3750–3753, 2022. Citado 4 vezes nas páginas 11, 20, 26 e 27.

MongoDB, Inc. *mongoimport — Database Tools*. 2026. <<https://www.mongodb.com/docs/database-tools/mongoimport/>>. Utilitário de importação de CSV/JSON/TSV para o MongoDB. Citado na página 24.

Neo4j, Inc. *Import — Operations Manual*. 2026. <<https://neo4j.com/docs/operations-manual/current/tools/neo4j-admin/neo4j-admin-import/>>. Importação em massa de CSV com neo4j-admin database import. Citado na página 25.

PEREIRA, F. d. O.; GUERRA, E.; ROSA, R. R. A framework model for supporting transparent polyglot persistence with a unified API and extensible for different database types. In: *Proceedings of the 27th International Conference on Enterprise Information Systems (ICEIS)*. [S.l.]: SciTePress, 2025. p. 109–120. Citado na página 26.

Redis Ltd. *Bulk Loading*. 2026. <<https://redis.io/docs/latest/develop/clients/patterns/bulk-loading/>>. Carga em massa via redis-cli --pipe usando o protocolo RESP. Citado na página 23.

ROY-HUBARA, N.; SHOVAL, P.; STURM, A. Selecting databases for polyglot persistence applications. *Data & Knowledge Engineering*, v. 137, p. 101950, 2022. Citado 2 vezes nas páginas 20 e 26.

SACHDEVA, S.; VASAVA, J. Plugging and playing with variety of data using multi-model database and polyglot persistence. In: *2024 Third International Conference on Electrical, Electronics, Information and Communication Technologies (ICEEICT)*. [S.l.]: IEEE, 2024. p. 1–8. Citado na página 26.

SADALAGE, P. J.; FOWLER, M. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. [S.l.]: Pearson Education, 2013. Citado 2 vezes nas páginas 11 e 20.

SILVA, H. A. B. d.; BORNIA, L. G.; MELLO, R. d. S. Um estudo sobre modelagem poliglota de dados. In: *Anais da Escola Regional de Banco de Dados (ERBD)*. [S.l.]: Sociedade Brasileira de Computação, 2024. p. 11–20. Citado 2 vezes nas páginas 11 e 26.

TAN, R. et al. Enabling query processing across heterogeneous data models: A survey. In: *2017 IEEE International Conference on Big Data (BigData)*. [S.l.: s.n.], 2017. p. 3211–3220. Citado 2 vezes nas páginas 20 e 26.

The Apache Software Foundation. *sstableloader*. 2026. <<https://cassandra.apache.org/doc/latest/cassandra/managing/tools/sstable/sstableloader.html>>. Ferramenta de carga em massa de SSTables no Apache Cassandra. Citado na página 24.

The PostgreSQL Global Development Group. *PostgreSQL Documentation: COPY*. 2026. <<https://www.postgresql.org/docs/current/sql-copy.html>>. Comando COPY para importação/exportação de dados, inclusive CSV. Citado na página 22.

YE, F. et al. A benchmark for performance evaluation of a multi-model database vs. polyglot persistence. *Journal of Database Management*, v. 34, n. 3, p. 1–20, 2023. Citado 2 vezes nas páginas 11 e 14.

APÊNDICES

APÊNDICE A – JSON SCHEMAS DE CONFIGURAÇÃO NA FORMA SERIALIZADA

Este apêndice reproduz os dois documentos *JSON Schema* (rascunho 2020-12) tal como embutidos em `src/polyglotimportcsv/schemas/`. A estrutura de alto nível de ambos é descrita na seção 4.2; os fragmentos por SGBD são apresentados nas respectivas subseções do capítulo 4.

A.1 SCHEMA DE IMPORTAÇÃO (IMPORT_CONFIG.SCHEMA.JSON)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://polyglot-import-csv.local/schemas/import_config.schema.json",
  "title": "PolyglotImportCSV import configuration",
  "description": "Declarative mapping from a wide CSV file to entities and
    relationships of one or more SGBDs. Connection settings live in the separate
    SGBD configuration.",
  "type": "object",
  "required": ["version"],
  "properties": {
    "version": {
      "type": "integer",
      "minimum": 1,
      "description": "Configuration format version."
    },
    "postgres": { "$ref": "#/$defs/postgresMapping" },
    "mongodb": { "$ref": "#/$defs/mongoMapping" },
    "cassandra": { "$ref": "#/$defs/cassandraMapping" },
    "redis": { "$ref": "#/$defs/redisMapping" },
    "neo4j": { "$ref": "#/$defs/neo4jMapping" }
  },
  "additionalProperties": false,
  "$defs": {
    "columnSpec": {
      "type": "object",
      "description": "Leaf mapping from a CSV column to a destination field.",
      "properties": {
        "is_key": {
          "type": "boolean",
          "description": "Primary or merge key for the entity."
        }
      },
    },
    "csv_column": {
      "description": "CSV header name or 0-based column index.",
      "oneOf": [
        { "type": "string", "minLength": 1 },
        { "type": "integer", "minimum": 0 }
      ]
    },
    "schema_column": {
      "type": "string",
      "minLength": 1,
    }
  }
}
```

```

        "description": "Destination field name when it differs from the JSON key."
    },
    "db_type": {
        "type": "string",
        "description": "SQL/CQL type for DDL generation (PostgreSQL, Cassandra)."
    }
},
"additionalProperties": false
},
"columnEntry": {
    "description": "Either a leaf columnSpec or a nested object (MongoDB
        subdocuments).",
    "oneOf": [
        { "$ref": "#/$defs/columnSpec" },
        {
            "type": "object",
            "minProperties": 1,
            "additionalProperties": { "$ref": "#/$defs/columnEntry" }
        }
    ]
},
"filter": {
    "type": "object",
    "required": ["column", "operator"],
    "properties": {
        "column": { "type": "string", "minLength": 1 },
        "operator": {
            "type": "string",
            "enum": ["=", "!=", ">", "<", ">=", "<=", "in", "not_in", "each"]
        },
        "value": {},
        "target_suffix": { "type": "string" }
    },
    "additionalProperties": false
},
"entity": {
    "type": "object",
    "required": ["columns"],
    "properties": {
        "columns": {
            "type": "object",
            "minProperties": 1,
            "additionalProperties": { "$ref": "#/$defs/columnEntry" }
        },
        "filters": {
            "type": "array",
            "items": { "$ref": "#/$defs/filter" }
        },
        "cassandra_partition": {
            "type": "array",
            "items": { "type": "string" }
        },
        "cassandra_cluster": {
            "type": "array",
            "items": { "type": "string" }
        }
    }
}

```

```
    },
    "additionalProperties": false
  },
  "postgresMapping": {
    "type": "object",
    "required": ["entities"],
    "properties": {
      "entities": {
        "type": "object",
        "additionalProperties": { "$ref": "#/$defs/entity" }
      },
      "relationships": {
        "type": "object",
        "additionalProperties": {
          "type": "object",
          "required": ["from", "to", "foreign_key"],
          "properties": {
            "from": { "type": "string" },
            "to": { "type": "string" },
            "foreign_key": { "type": "string" },
            "references_key": { "type": "string" }
          },
          "additionalProperties": false
        }
      }
    },
    "additionalProperties": false
  },
  "mongoMapping": {
    "type": "object",
    "required": ["entities"],
    "properties": {
      "entities": {
        "type": "object",
        "additionalProperties": { "$ref": "#/$defs/entity" }
      }
    },
    "additionalProperties": false
  },
  "cassandraMapping": {
    "type": "object",
    "required": ["entities"],
    "properties": {
      "entities": {
        "type": "object",
        "additionalProperties": { "$ref": "#/$defs/entity" }
      }
    },
    "additionalProperties": false
  },
  "redisMapping": {
    "type": "object",
    "required": ["entities"],
    "properties": {
      "entities": {
        "type": "object",
```

```

        "additionalProperties": { "$ref": "#/$defs/entity" }
    },
    "additionalProperties": false
},
"neo4jMapping": {
    "type": "object",
    "required": ["entities"],
    "properties": {
        "entities": {
            "type": "object",
            "additionalProperties": { "$ref": "#/$defs/entity" }
        },
        "relationships": {
            "type": "object",
            "additionalProperties": {
                "type": "object",
                "required": ["from", "to", "type"],
                "properties": {
                    "from": { "type": "string" },
                    "to": { "type": "string" },
                    "type": { "type": "string" },
                    "columns": {
                        "type": "object",
                        "additionalProperties": { "$ref": "#/$defs/columnSpec" }
                    }
                }
            },
            "additionalProperties": false
        }
    },
    "additionalProperties": false
}
}
}

```

A.2 SCHEMA DE CONEXÃO (SGBD_CONFIG.SCHEMA.JSON)

```

{
    "$schema": "https://json-schema.org/draft/2020-12/schema",
    "$id": "https://polyglot-import-csv.local/schemas/sghd_config.schema.json",
    "title": "PolyglotImportCSV SGBD connection configuration",
    "description": "Connection settings for each SGBD. Lists which SGBDs are available
        ; the import configuration may only target SGBDs declared here.",
    "type": "object",
    "required": ["version"],
    "properties": {
        "version": { "type": "integer", "minimum": 1 },
        "postgres": { "$ref": "#/$defs/postgresConnection" },
        "mongodb": { "$ref": "#/$defs/mongoConnection" },
        "cassandra": { "$ref": "#/$defs/cassandraConnection" },
        "redis": { "$ref": "#/$defs/redisConnection" },
        "neo4j": { "$ref": "#/$defs/neo4jConnection" }
    },
    "additionalProperties": false,
}

```

```
"$defs": {
  "postgresConnection": {
    "type": "object",
    "properties": {
      "connection": {
        "type": "object",
        "properties": {
          "host": { "type": "string" },
          "port": { "type": "integer" },
          "database": { "type": "string" },
          "user": { "type": "string" },
          "password": { "type": "string" }
        }
      },
      "additionalProperties": false
    },
    "schema": {
      "type": "string",
      "default": "public"
    }
  },
  "additionalProperties": false
},
"mongoConnection": {
  "type": "object",
  "properties": {
    "connection": {
      "type": "object",
      "properties": {
        "uri": { "type": "string" },
        "database": { "type": "string" }
      }
    },
    "required": ["uri", "database"],
    "additionalProperties": false
  }
},
"additionalProperties": false
},
"cassandraConnection": {
  "type": "object",
  "properties": {
    "connection": {
      "type": "object",
      "properties": {
        "hosts": { "type": "array", "items": { "type": "string" } },
        "port": { "type": "integer" },
        "keyspace": { "type": "string" },
        "protocol_version": { "type": "integer" }
      }
    },
    "required": ["hosts", "keyspace"],
    "additionalProperties": false
  }
},
"additionalProperties": false
},
"redisConnection": {
  "type": "object",
```

```
"properties": {
  "connection": {
    "type": "object",
    "properties": {
      "host": { "type": "string" },
      "port": { "type": "integer" },
      "db": { "type": "integer" },
      "password": { "type": "string" }
    },
    "additionalProperties": false
  }
},
"additionalProperties": false
},
"neo4jConnection": {
  "type": "object",
  "properties": {
    "connection": {
      "type": "object",
      "properties": {
        "uri": { "type": "string" },
        "user": { "type": "string" },
        "password": { "type": "string" },
        "database": { "type": "string" }
      },
      "required": ["uri", "user", "password"],
      "additionalProperties": false
    }
  },
  "additionalProperties": false
}
}
```

APÊNDICE B – O ARQUIVO DE DADOS ECOMMERCE_JOIN.CSV

Este apêndice reproduz, em forma tabular, o arquivo de dados do cenário e-commerce, `data/ecommerce/ecommerce_join.csv`, utilizado nas seções 4.3 e 4.3.3. O arquivo contém 32 linhas de dados e 39 colunas, e resulta da **junção** (*join*) de todas as coleções de dados da fonte — usuários, endereços, produtos, categorias, estoque, pedidos, seleções de produto e carrinhos de compra — em uma única tabela larga. Por isso ele é **esparso**: cada linha representa um evento da loja virtual, identificado pela coluna `action` (`stock`, `purchase`, `select_product` ou `add_to_cart`), e preenche apenas as colunas pertinentes àquele tipo de evento — as demais permanecem vazias. A Tabela 7 resume quais grupos de colunas cada ação preenche.

Tabela 7 – Grupos de colunas do ecommerce_join.csv preenchidos por ação.

Grupo	Colunas	<i>stock</i>	<i>purchase</i>	<i>select_product</i>	<i>add_to_cart</i>
Evento	action, timestamp	•	•	•	•
Usuário	user_id, user_name, user_email	•	•	•	•
Endereço do usuário	street, neighborhood, state, country, zip_code	•	•	—	—
Pedido	order_number, order_date, order_status	—	•	—	—
Contraparte da venda	traded_with, trader_street, trader_neighborhood, trader_state, trader_country, trader_zip_code	—	•	—	—
Avaliação e pagamento	comment, rating, payment_method, quantity	—	•	—	—
Produto	product_id, product_name, product_variant, product_brand, product_description, product_image	•	•	—	—
Categoria	category_id, category_name	•	•	—	—
Estoque	quantity_available, price, last_restock_date	•	•	—	—
Seleção de produto	selected_product_id, suggested_product_count	—	—	•	—
Carrinho	shopping_cart_id, cart_product_id, cart_quantity	—	—	—	•

Fonte: Elaborado pelo autor (2026).

Vários valores do arquivo seguem padrões regulares, o que permite apresentá-

los de forma resumida sem perda de informação. O produto $\text{Product}N$ tem $\text{product_id } N$, pertence à categoria $\text{Categoria}N$ ($\text{category_id } N$) e tem descrição e imagem padronizadas ($\text{Description}M$ e <https://example.com/imageM.jpg>). Cada usuário $\text{user}N$ tem e-mail $\text{user}N\text{@example.com}$, rua $\text{Street } N$, 42 e um identificador alfanumérico (user_id) cujo início é dado na Tabela 8, junto com os demais dados de endereço. Nas tabelas deste apêndice, os usuários são referidos pelo nome ($\text{user}N$), tanto na coluna user_id quanto em traded_with .

Tabela 8 – Usuários do ecommerce_join.csv e seus dados de identificação e endereço.

Usuário	user_id (início)	Bairro	Estado	País	CEP
user1	7fd7eee7-. . .	ABC	CA	United States	12345
user2	f246fb91-. . .	DEF	SC	Brasil	84102149
user3	09b3458a-. . .	GHI	RS	Brasil	18057313
user4	1fd3d09a-. . .	JKL	TO	Brasil	19645442
user5	0331463b-. . .	MNO	RJ	Brasil	30097508
user6	f4857da9-. . .	PQR	SP	Brasil	92083256
user7	a87df805-. . .	STU	SP	Brasil	67065799
user8	c11a1ef2-. . .	VWX	SE	Brasil	18439703
user9	d0416923-. . .	YZ	AM	Brasil	18439666

Fonte: Elaborado pelo autor (2026).

Nas seções seguintes, o arquivo é apresentado em quatro blocos, um por ação. Cada tabela exibe as colunas com valores próprios daquele bloco; as colunas cujo valor é constante ou segue os padrões acima são descritas no texto que antecede cada tabela. Os valores e a ordem das linhas são exatamente os do arquivo original.

B.1 LINHAS COM ACTION == STOCK

As oito linhas de reposição de estoque preenchem os grupos de usuário, endereço, produto, categoria e estoque (21 das 39 colunas). Todas ocorrem em 02/11/2023 e, em cada uma, last_restock_date repete o instante do próprio evento (timestamp). A Tabela 9 apresenta os valores.

Tabela 9 – Linhas action == stock do ecommerce_join.csv.

Hora	Usuário	Produto	Variante	Marca	Categoria	Qtd.	Preço
03:30	user1	Product2	blue	Microsoft	Categoria2	2	50.5
06:45	user2	Product3	green	Google	Categoria3	1	52.07
09:15	user2	Product9	black	Pepsi	Categoria9	67	32.78
12:00	user8	Product8	white	Google	Categoria8	60	46.79
14:30	user9	Product4	yellow	Apple	Categoria4	82	55.94
17:15	user9	Product5	red	Samsung	Categoria5	57	34.21
20:00	user9	Product6	orange	Coca-Cola	Categoria6	64	51.98
22:30	user9	Product7	grey	Audi	Categoria7	1	100000

Fonte: Elaborado pelo autor (2026).

B.2 LINHAS COM ACTION == PURCHASE

As oito linhas de compra são as mais completas do arquivo (34 das 39 colunas): acrescentam aos grupos anteriores os dados de pedido, contraparte da venda, avaliação e pagamento. Em todas elas, order_status vale Completed e quantity vale 1; o registro do evento (timestamp) ocorre entre 15 e 19/11/2023, posterior à data do pedido (order_date), que inclui hora e fuso horário no arquivo original. O pagamento (payment_method) é cash nos pedidos ORD1 e ORD2 e credit_card nos demais. As colunas de produto, categoria e estoque trazem os dados cadastrais do produto negociado, tal como nas linhas de estoque (Tabela 9); o endereço do comprador e o do vendedor (colunas trader_*) são os da Tabela 8. A Tabela 10 apresenta os valores próprios de cada compra.

Tabela 10 – Linhas action == purchase do ecommerce_join.csv.

Pedido	Data	Comprador	Vendedor	Produto	Nota	Comentário
ORD1	03/11	user1	user2	Product2	2	Bad
ORD2	03/11	user2	user1	Product3	4	Bom
ORD3	05/11	user3	user9	Product4	3	Médio
ORD4	07/11	user4	user9	Product5	5	Ótimo produto
ORD5	09/11	user5	user9	Product6	1	Não gostei
ORD6	10/11	user6	user9	Product7	5	—
ORD7	11/11	user7	user8	Product8	1	Reprovado
ORD8	14/11	user8	user2	Product9	4	Bom, bonito e barato

Fonte: Elaborado pelo autor (2026).

B.3 LINHAS COM ACTION == SELECT_PRODUCT

As oito linhas de seleção de produto preenchem apenas a identificação do evento, o usuário e o grupo de seleção (7 das 39 colunas): o produto visualizado (selected_product_id) e a quantidade de sugestões exibidas (suggested_product_count), que vale 3 em todas as linhas. A Tabela 11 apresenta os valores.

Tabela 11 – Linhas action == select_product do ecommerce_join.csv.

Momento	Usuário	Produto selecionado	Sugestões
15/11 15:09	user1	11	3
16/11 03:11	user2	12	3
16/11 17:50	user3	22	3
17/11 06:05	user4	14	3
17/11 21:56	user5	14	3
18/11 10:09	user6	21	3
18/11 23:20	user7	33	3
19/11 15:18	user8	17	3

Fonte: Elaborado pelo autor (2026).

B.4 LINHAS COM ACTION == ADD_TO_CART

As oito linhas de adição ao carrinho preenchem a identificação do evento, o usuário e o grupo de carrinho (8 das 39 colunas). O identificador do carrinho (shopping_cart_id) segue o padrão user:<user_id>:cart — por exemplo, user:7fd7eee7-. . . :cart para user1. A Tabela 12 apresenta os demais valores.

Tabela 12 – Linhas action == add_to_cart do ecommerce_join.csv.

Momento	Usuário	Produto adicionado	Quantidade
15/11 15:17	user1	3	1
16/11 03:27	user2	2	1
16/11 18:08	user3	2	1
17/11 06:14	user4	2	1
17/11 22:11	user5	2	1
18/11 10:21	user6	2	1
18/11 23:34	user7	2	1
19/11 15:29	user8	2	1

Fonte: Elaborado pelo autor (2026).

APÊNDICE C – ARQUIVOS DE CONFIGURAÇÃO DO CENÁRIO E-COMMERCE

Este apêndice reproduz na íntegra os dois arquivos de configuração do cenário de e-commerce utilizado no capítulo 4, tal como se encontram no diretório `data/ecommerce/` do repositório da ferramenta: o arquivo de importação `import_config.json` (Listagem C.1), cujos fragmentos por SGBD ilustraram a seção 4.2, e o arquivo de conexão `sgbd_config.json` (Listagem C.2). Juntos com o arquivo de dados reproduzido no Apêndice B, eles constituem as três entradas completas do exemplo, e neles podem ser conferidas as entidades, filtros e chaves que fundamentam as saídas materializadas da seção 4.3.2 e as contagens da Tabela 6.

C.1 ARQUIVO DE IMPORTAÇÃO (IMPORT_CONFIG.JSON)

```
{
  "version": 1,

  "postgres": {
    "entities": {
      "categories": {
        "columns": {
          "category_id": { "is_key": true, "db_type": "BIGINT" },
          "category_name": { "db_type": "TEXT" }
        },
        "filters": []
      },
      "products": {
        "columns": {
          "product_id": { "is_key": true, "db_type": "BIGINT" },
          "product_name": { "db_type": "TEXT" },
          "product_variant": { "db_type": "TEXT" },
          "product_brand": { "db_type": "TEXT" },
          "category_id": { "db_type": "BIGINT" },
          "price": { "db_type": "NUMERIC(14,2)" }
        },
        "filters": []
      },
      "inventory": {
        "columns": {
          "product_id": { "is_key": true, "db_type": "BIGINT" },
          "quantity_available": { "db_type": "BIGINT" },
          "last_restock_date": { "db_type": "TIMESTAMPZ" },
          "price": { "db_type": "NUMERIC(14,2)" }
        },
        "filters": [{ "column": "action", "operator": "=", "value": "stock" }]
      },
      "orders": {
        "columns": {
          "order_number": { "is_key": true, "db_type": "TEXT" },
          "user_id": { "db_type": "TEXT" },
```

```

        "order_date": { "db_type": "TIMESTAMP TZ" },
        "order_status": { "db_type": "TEXT" },
        "payment_method": { "db_type": "TEXT" },
        "quantity": { "db_type": "BIGINT" },
        "product_id": { "db_type": "BIGINT" },
        "price": { "db_type": "NUMERIC(14,2)" },
        "comment": { "db_type": "TEXT" },
        "rating": { "db_type": "BIGINT" }
    },
    "filters": [{ "column": "action", "operator": "==", "value": "purchase" }]
}
},
"relationships": {
    "product_category": {
        "from": "products",
        "to": "categories",
        "foreign_key": "category_id",
        "references_key": "category_id"
    },
    "order_product": {
        "from": "orders",
        "to": "products",
        "foreign_key": "product_id",
        "references_key": "product_id"
    }
}
},
"mongodb": {
    "entities": {
        "product_catalog": {
            "columns": {
                "product_id": {},
                "product_name": {},
                "product_variant": {},
                "product_brand": {},
                "product_description": {},
                "product_image": {},
                "price": {},
                "category": {
                    "category_id": {},
                    "category_name": {}
                }
            },
            "stock": {
                "quantity_available": {},
                "last_restock_date": {}
            }
        },
        "filters": [{ "column": "action", "operator": "==", "value": "stock" }]
    }
}
},
"cassandra": {
    "entities": {
        "user_activity_log": {

```

```

    "columns": {
      "user_id": {},
      "timestamp": { "schema_column": "event_time" },
      "action": { "schema_column": "event_type" },
      "product_id": {},
      "order_number": {},
      "selected_product_id": {},
      "shopping_cart_id": {}
    },
    "cassandra_partition": ["user_id"],
    "cassandra_cluster": ["timestamp"],
    "filters": []
  }
}
},

"redis": {
  "entities": {
    "shopping_cart": {
      "columns": {
        "shopping_cart_id": { "is_key": true },
        "cart_product_id": {},
        "cart_quantity": {}
      },
      "filters": [{ "column": "action", "operator": "==", "value": "add_to_cart"
        }]
    },
    "user_session": {
      "columns": {
        "user_id": { "is_key": true },
        "user_name": {},
        "user_email": {},
        "timestamp": { "schema_column": "last_seen" }
      },
      "filters": [{ "column": "action", "operator": "==", "value": "select_product
        " }]
    }
  }
},

"neo4j": {
  "entities": {
    "User": {
      "columns": {
        "user_id": { "is_key": true },
        "user_name": {},
        "user_email": {}
      },
      "filters": [{ "column": "action", "operator": "==", "value": "purchase" }]
    },
    "Product": {
      "columns": {
        "product_id": { "is_key": true },
        "product_name": {},
        "product_brand": {}
      },

```

```
    "filters": [{ "column": "action", "operator": "==", "value": "stock" }]
  },
  "relationships": {
    "PURCHASED": {
      "from": "User",
      "to": "Product",
      "type": "PURCHASED",
      "columns": {
        "order_number": { "is_key": true },
        "quantity": {},
        "price": {},
        "rating": {}
      }
    }
  }
}
```

Listagem C.1 – Arquivo de importação completo do cenário e-commerce (import_config.json).

C.2 ARQUIVO DE CONEXÃO (SGBD_CONFIG.JSON)

```
{
  "version": 1,

  "postgres": {
    "connection": {
      "host": "127.0.0.1",
      "port": 5432,
      "database": "ecommerce",
      "user": "postgres",
      "password": "postgres"
    },
    "schema": "public"
  },

  "mongodb": {
    "connection": {
      "uri": "mongodb://127.0.0.1:27017",
      "database": "ecommerce"
    }
  },

  "cassandra": {
    "connection": {
      "hosts": ["127.0.0.1"],
      "port": 9042,
      "keyspace": "ecommerce"
    }
  },

  "redis": {
    "connection": { "host": "127.0.0.1", "port": 6379, "db": 0 }
```

```
},  
  
"neo4j": {  
  "connection": {  
    "uri": "bolt://127.0.0.1:7687",  
    "user": "neo4j",  
    "password": "password",  
    "database": "neo4j"  
  }  
}  
}
```

Listagem C.2 – Arquivo de conexão completo do cenário e-commerce (sgbd_config.json).